AI and Technical Debt: Evidence From 4 Billion Lines of Code*

João Dimas† Y. Eddie Lu‡ Labros Pnevmatikos§

November 9, 2026

Abstract

We examine whether generative artificial intelligence (AI) speeds up software development at the expense of code quality. Our novel monthly panel tracks 1,459 corporate public software projects from 2022 to 2026, covering 3.9 billion lines of source code. An industry-standard tool identifies maintainability issues, bugs, and security vulnerabilities in each monthly snapshot of the code. We convert its estimated repair time into a dollar measure of technical debt, the future cost of resolving maintainability issues. We compare code quality changes around first detected AI use with changes at comparable projects where we have not yet detected AI use. Contrary to concerns about "AI slop," total issues per line decline by 1.4%, bugs per line by 2.1%, and technical debt per line by 0.7% of their averages before detection. Quality gains concentrate in projects with weaker development practices before AI use. We also find that issues per unit of software functionality decline relative to comparison projects. Together, the findings link detected AI use to faster growth in firms' software assets and a lower engineering burden relative to both code volume and software features.

JEL Classification: O33, L86, D22, J24, E22
Keywords: Artificial intelligence, productivity, intangible assets, technical debt, software quality

*We acknowledge an in-kind research grant from Sonar. The grant agreement preserves the authors' independence over methodology, findings, and outputs.

†Olin Business School at Washington University in St. Louis. Send correspondence to João Dimas; e-mail: joaodimas@wustl.edu.
‡Olin Business School at Washington University in St. Louis.
§Olin Business School at Washington University in St. Louis.

1 Introduction

An important question about generative artificial intelligence (AI) is whether the productivity gains it brings translate into greater value for firms. Experimental and field studies document substantial gains as workers use AI to complete tasks faster and produce more output (Noy and Zhang, 2023; Brynjolfsson, Li, and Raymond, 2025; Peng et al., 2023; Cui et al., 2026). The value of this output, however, also depends on its quality. Work that is cheaper to produce with AI may still be costly to verify, correct, and maintain. Industry research documents the spread of "AI slop", AI-generated work that looks good but lacks substance, and 40% of U.S. desk workers report receiving it in the previous month (BetterUp Labs and Stanford Social Media Lab, 2025). Firms have already faced financial consequences over fabricated citations and other defects in delivered AI-assisted work (U.S. District Court, 2023; Associated Press, 2025). Existing studies measure how much faster AI lets workers produce, but far less is known about the quality of what they produce. Does the speed that AI brings come at the cost of quality?

We study this question in software, a durable asset whose quality shapes its future costs to the firm. Firms must understand, maintain, and build on their code as their needs change. To deliver working code quickly, developers and AI coding tools can take shortcuts, such as duplicating code or writing needlessly complex logic. These shortcuts make later changes more costly. The future cost of remediating such shortcuts is known as technical debt (Cunningham, 1992). It does not appear in financial statements, but firms pay for it. Chief information officers at large firms estimate that technical debt amounts to 20–40% of the value of their technology assets, and developers report spending about a third of the work week addressing it (McKinsey & Company, 2020; Stripe, Inc., 2018).

Whether AI raises or lowers this liability is an open question. AI could reduce it by helping developers verify, document, and change what they produce. It could also raise it, because faster production leaves firms with more to maintain and with work that no one has fully verified. Benchmark evidence points to the second possibility. More capable models write more code, and more complex code, to solve the same programming tasks (Sarkar, 2025). Benchmarks, however, score what a model writes on its own for a self-contained task. Whether quality deteriorates in practice depends on how developers use these tools, how much of their output is reviewed and revised, and what remains in the codebase over time.

This tension extends to the intangible capital that firms increasingly build with AI, such as software, databases, and product designs (Crouzet et al., 2022). The quality of these assets is hard to study. Firms do not disclose it, the assets are usually proprietary, and defects surface only later, as rework, failures, or legal claims. Public software is an exception. Corporate public projects expose their full development history.

We analyze 3.9 billion lines of code across 42,187 monthly codebase snapshots from 1,459

corporate public projects. These projects are repositories on GitHub, an online platform for storing code and coordinating its development. We keep the repositories whose owners list a business website. Firms keep public projects to support their products and infrastructure, and these projects are often economically important to them (Lerner and Tirole, 2002; Nagle, 2018; Conti, Peukert, and Roche, 2025; Emery, Lim, and Ye, 2025). Each snapshot is evaluated with SonarQube, an industry-standard tool that checks code against thousands of fixed rules. Unlike an AI model asked to judge code, it always produces the same results for the same code, and because we hold its version fixed, the same standard applies throughout our sample. The tool identifies maintainability issues, which make code harder to modify or extend, bugs, which can cause the software to fail when it runs, and security vulnerabilities, which attackers could exploit. We refer to the three together as total issues.

The development records of each project log every change to the code as a commit, together with a message in which the developer describes what changed and why. The same records show when developers use AI tools. Coding assistants leave configuration files in the code, commit messages list AI tools as coauthors, and coding agents propose their own code changes for others to review. First detected AI use is the month in which a project's records first show any of these signals. In the months we study, developers contribute more than 1.1 million commits to these projects.

We estimate a stacked difference-in-differences model around first detected AI use. Comparison projects are those in which we detect no AI use during the estimation window, either because we detect it later or never do. Projects do not adopt AI at random. Before detection, the projects where we detect AI use are larger and have fewer total issues and less technical debt per line than comparison projects. We therefore weight the comparison projects with entropy balancing (Hainmueller, 2012). Within each monthly cohort, weighting aligns the two groups' paths over the year before detection in code size, total issues per line, and technical debt per line. Because codebases grow, we measure issues and technical debt per line of code and express changes as percentages of their averages before detection.

Codebase growth is approximately 6.4 percentage points greater than in comparison projects. Total issues per line, most of which are maintainability issues, decline by 1.4%, and bugs per line decline by 2.1%. In our dynamic difference-in-differences specification, the gaps in code size, total issues, and technical debt appear only after detection, with no evidence of diverging trends beforehand, and the gap in code size widens over time.

As with financial debt, technical debt has a principal amount: the estimated cost of correcting maintainability issues. SonarQube estimates the time required for these corrections, and we convert it into dollars. Relative to comparison projects, technical debt per line declines by 0.7% of its average before detection. Among projects with high initial technical debt per line, the decline is 2.1%.

Firms value software for what it enables them to do. Lines of code measure implementation volume, but functionality per line can change. Consequently, stable technical debt per line need not imply stable technical debt relative to the software asset. We measure functionality by counting

commits labeled as new features. Growth in total issues per unit of functionality is approximately 6.7 percentage points lower than in comparison projects. Growth in technical debt per unit of functionality is approximately 5.0 percentage points lower. Alternative weights and samples give similar growth differences. These results extend the quality comparison from the amount of code to recorded functional development.

Projects that adopt AI may differ from other projects, and their first AI use may coincide with other changes. Such differences could explain faster growth and better quality even without AI. We address this concern in two ways. First, we restrict the comparison group to projects where we detect later AI usage, so both groups eventually show evidence of AI use. In this comparison, codebase growth is approximately 3.3 percentage points greater, and total issues and bugs per line decline by 1.5% and 2.3%, close to the main estimates. Second, we estimate synthetic difference-in-differences (Arkhangelsky et al., 2021). This estimator weights both projects and months to match the paths of fourteen project characteristics before detection, including commits and commit authors, and uses the same weights for every outcome. Under this estimator, we also find faster code growth and fewer total issues and bugs per line against both comparison groups.

We also examine whether investor financing accounts for these changes. A funding round may bring both AI tools and resources for improving software. We repeat the comparison after excluding projects whose companies have recorded equity funding. Among the 296 projects with recorded AI use that remain, codebases still grow faster and total issues, bugs, and technical debt per line decline. The pattern also holds when we keep only projects linked to companies in the financing data.

AI coding tools could also leave firms with more duplicated or more complex code, the shortcuts described above. We find no evidence of a change in the duplicated-code share or in complexity per line. The same holds for critical issues, the most severe ones, for security vulnerabilities, and for security-review flags, which mark code that needs manual security review.

Quality gains concentrate in projects with weaker development practices before AI use. We measure these practices using human review, the integration of proposed changes, testing, and configured automatic checks. In the weaker half, total issues per line decline by 2.1%, bugs by 3%, and technical debt by 1.8% of their pre-detection averages. We find no corresponding quality gains in the stronger half. Testing provides one channel for this catch-up. Test files accumulate in the weaker half, and AI-marked contributions are more likely to include tests than unmarked contributions within the same project and month. Holding the change in test-file share fixed reduces the weaker half's issue decline by about one fifth. These findings suggest that AI can help firms add checks where development practices were initially less established, alongside expanding their software assets.

We then examine development activity. After first detected AI use, projects open more human-authored pull requests, the documented proposals to change the code. This increase holds against both comparison groups. A larger share of their commits concerns security or maintenance work such as fixes. This shift toward security and fix work accompanies the declines in total issues and bugs per

line of code.

Our results contribute to two strands of the literature. First, we add to research on how generative AI changes what firms produce. Experimental and field studies document productivity gains in writing, customer support, consulting, and software development (Noy and Zhang, 2023; Brynjolfsson, Li, and Raymond, 2025; Dell'Acqua et al., 2026; Peng et al., 2023; Cui et al., 2026). AI tools also shift developers toward core coding work (Hoffmann et al., 2024). Other studies ask how AI's task-level gains add up to output, employment, and firm value (Acemoglu, 2025; Eloundou et al., 2024; Humlum and Vestergaard, 2025; Hampole et al., 2025; Babina et al., 2024; Eisfeldt, Schubert, and Zhang, 2023). Most of this work measures how fast workers complete tasks and how much they produce. Where it examines quality, it rates individual tasks. We examine the quality of the software asset itself by scanning the full codebase of each corporate project around first detected AI use. We find that output rises while the quality of this asset does not deteriorate.

Closest to our paper, two recent studies examine open-source projects around a single AI tool or channel. He et al. (2026) study first visible use of Cursor, an AI coding tool, and Agarwal, He, and Vasilescu (2026) study a project's first agent-generated pull request. Both report higher totals of issues and complexity, with gains in development speed that fade or appear only in some projects. We complement this work in three ways. We focus on corporate software, which firms rely on and have a stake in. We follow the first signal of AI use across tools and channels. And we measure quality per line of code, which separates growth in the amount of software from changes in the frequency of issues within it. Total complexity rises in our sample as well, but code growth alone predicts most of this rise, and we find no evidence that complexity per line changes. For firms, the evidence points to growth in their software assets without more issues per line of code. We also measure engineering quality per unit of functionality, a proxy for the value of firms' software assets.

Second, we add to research on intangible capital (Crouzet et al., 2022). AI is changing how firms produce this capital, and the quality of what they produce is rarely visible. Prior work links technical debt to firm performance and engineering quality to how a firm is governed (Banker, Liang, and Ramasubbu, 2021; Dimas and Pnevmatikos, 2026). We show how engineering quality changes with the technology that firms use to build software. Like deferred capital expenditure, technical debt can speed up development in the short term but raises the cost of maintaining the asset later. We find no evidence that firms defer more of this cost per line of code after first detected AI use. Software capital accumulates faster, and technical debt per line does not rise. Where projects start with high technical debt, it falls. The concentration of quality gains among projects with weaker initial practices identifies where these benefits are largest. The testing evidence connects them to investment in maintaining the software asset. Because the long-term value of AI's productivity gains depends on the quality of what firms produce, these results bear on the return firms earn on their AI investment.

The remainder of the paper proceeds as follows. Section 2 describes the data, sample construction, and measures of engineering quality, technical debt, and AI use. Section 3 presents the empirical

design. Section 4 reports the main results, their robustness, the other quality measures, heterogeneity in initial technical debt and development practices, and the evidence on development activity and testing. Section 5 concludes.

2 Data and Descriptive Statistics

We construct a monthly panel of corporate public software projects and record the month in which we first detect AI use in each project. We obtain a snapshot of each project's complete codebase at the end of every month from October 2022 through July 2026. The panel combines four sources. Records of public GitHub repositories with at least 50 bookmarks (stars), together with their owners' email addresses and websites, identify accounts that are likely to be firms. Monthly scans of each full codebase measure engineering quality. Commit histories and the pull requests that coding agents open reveal AI assistance. Pull-request records also show the changes developers propose and how others review them. The sources cover different months, so each comparison uses the months in which its outcome is observed. Appendix A documents each construction step.

2.1 Software Repositories and Development

GitHub is an online platform on which developers store software and coordinate changes to it. Each project is organized in a *repository*, an archive containing the source code—the instructions that determine what the software does—and a history of recorded changes. Repositories can be public or private. Most firms keep their code private, but their public repositories let us follow the software and its development over time.

Firms usually maintain public projects to support products and infrastructure, attract outside contributions, and develop complementary tools (Lerner and Tirole, 2002; Nagle, 2018; Llanes and de Elejalde, 2013). Lyft's corporate GitHub account, for example, hosts *cartography*, a tool its engineers developed and maintain to map the company's cloud infrastructure. It is the account's most-starred repository. Firms often draw value from such projects (Greenstein and Nagle, 2014; Hoffmann, Nagle, and Zhou, 2024; Conti, Peukert, and Roche, 2025; Emery, Lim, and Ye, 2025). Public projects also benefit the firms and other projects that build on them, whoever maintains them (Fershtman and Gandal, 2011; Nagle, 2019).

GitHub uses Git, a system that records successive versions of a project's files. A *commit* is a recorded set of changes, accompanied by an author identifier, a timestamp, and a message describing the work. Commits vary in size, and a commit need not correspond to a completed feature or a fixed amount of labor. The *commit history* chains these records. From it we can reconstruct the code present at any past date and measure quality before and after first detected AI use. Commit counts, authors, and messages also record development activity, and some messages disclose AI assistance.

Developers can also submit a *pull request*, a documented proposal to change the project's main codebase. Other developers can review the proposal, request revisions, or approve it. A *merge* applies an accepted proposal to the codebase. Other proposals stay open or are closed without a merge. A month-end snapshot of the code contains only what the project has retained by that date. It can already reflect revisions made during review, and a rejected proposal may never enter it.

From these records we measure development activity in each project and month. We count commits and human-authored pull requests opened. Among these pull requests, we also count those merged, those reviewed by at least one human, and the human requests for changes, each within thirty days of opening. The pull-request and review counts use only the months that the corresponding records cover. We measure the shares of commit messages that mention security, bug fixes, or testing. We also measure how often messages credit human or AI contributions through reviewer acknowledgments, sign-offs, or coauthor credits (Appendix A.4).

Software functionality. We measure functionality by counting commits labeled as new features. These labels include `feat:` and scoped forms such as `feat(api):`. These commits record additions to functionality; their cumulative count provides a proxy for the stock of functionality in the codebase. We divide total issues and technical debt in dollars by the cumulative number of these commits. Appendix E.1 describes the counting procedure, sample requirements, and log specification.

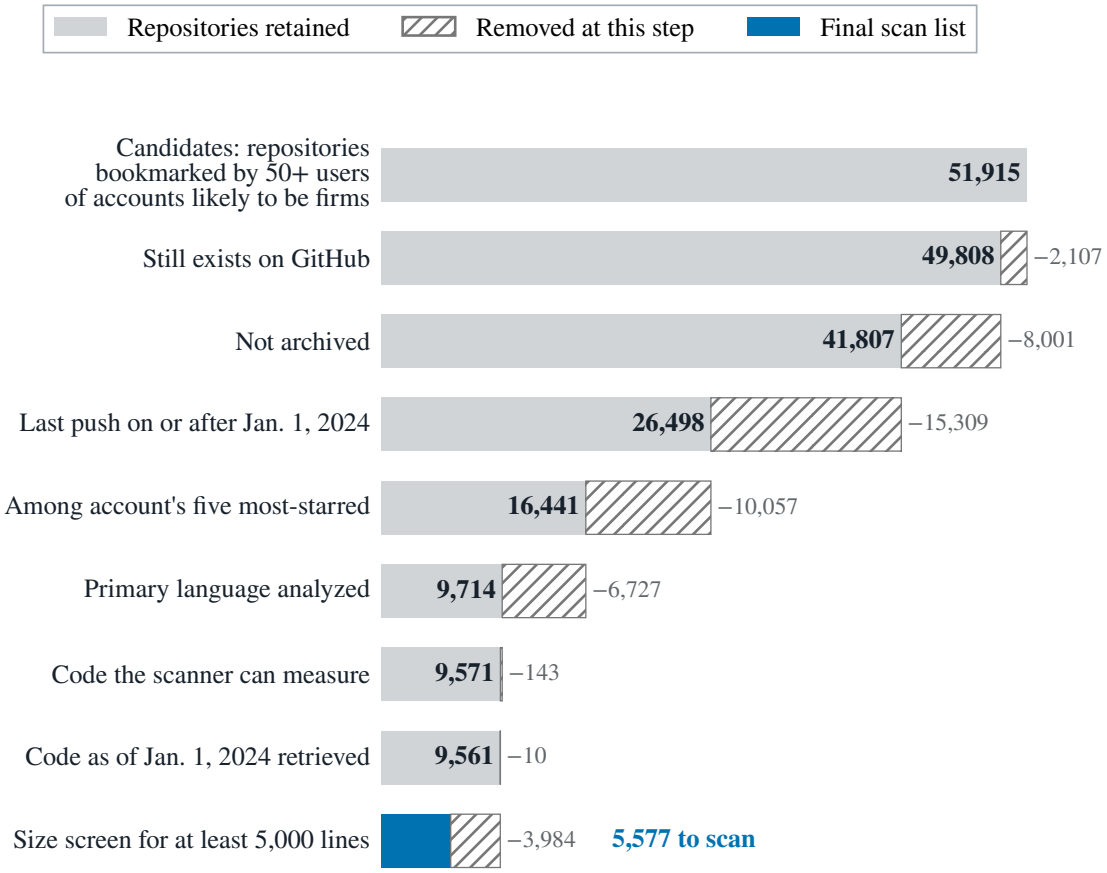
2.2 Sample Construction

Our universe consists of repositories owned by accounts whose websites show evidence of a business (Table 1 in Appendix A). From GitHub Archive, a public record of activity on GitHub, we take every public repository that at least 50 distinct users had bookmarked with a *star* by mid-February 2026. These 854,735 repositories belong to 415,627 accounts. After dropping accounts that no longer exist or whose listed website is a GitHub page or a social-media profile, we link 329,809 accounts to 680,613 repositories in our records by account name. The 13,511 accounts whose websites score at least four on signals of commercial activity, such as careers, pricing, and product pages, own 51,915 repositories that at least 50 users have bookmarked. These repositories are our candidates for scanning (Appendix A.1).

We apply eight filters to these candidates in a fixed order (Figure 1). We keep repositories that still exist on GitHub, are not archived, were last pushed on or after January 1, 2024, and rank among their account's five most-starred remaining repositories. The last-push filter and the cap of five repositories per account remove the most candidates. Our scanner must also be able to measure the code, so a repository must be written mostly in Python, JavaScript, TypeScript, Java, Go, PHP, Ruby, Scala, or Kotlin (Section 2.3). Finally, we screen repositories for at least 5,000 lines of code using

recorded size or pre-2024 line counts, with exceptions detailed in Appendix A.2. Every project must also have at least 5,000 lines in the month before each detection cohort it enters.

Figure 1: From Candidate Repositories to the Repositories We Scan



Notes: Each bar counts the repositories that remain after the filter in its row, with filters applied from top to bottom, and hatched segments mark the repositories each filter removes. Table 1 in Appendix A reports repository and account counts for every construction step, including the steps that identify the candidate repositories; Appendix A.2 explains the size screen.

The filters leave 5,577 repositories from 3,654 accounts. Eleven repositories enter twice, so 5,566 of them are distinct (Appendix A.2). GitHub registers the accounts behind 959 of the 5,577 as individual users rather than organizations. Because the filters use each repository's status as recorded in 2026, the repositories we scan represent corporate projects that are still public and unarchived in 2026. We exclude projects deleted or archived during our period. We obtain a full codebase snapshot for every month from October 2022, or the repository's creation month, through July 2026.

2.3 Measuring Engineering Quality

We measure engineering quality with SonarQube, a static-analysis tool that reads source code without running it and checks the code against a fixed set of deterministic rules for each programming language.

The rules encode common coding and security practices. Each rule that a piece of code violates produces an *issue*, the basic unit of our quality measures. Rules flag, for example, commented-out code, which obscures what the software actually does, and overly complex functions, which are harder to read, test, and modify. We group issues into three families. *Maintainability issues* make code harder to understand or modify, as when duplicated instructions force a later correction to be repeated in each place. *Bugs* are mistakes that can make the software fail or behave unexpectedly when it runs. The security family covers *security vulnerabilities*, weaknesses that an attacker could exploit, and *security-review flags*, which mark sensitive code for manual review. *Total issues* count maintainability issues, bugs, and security vulnerabilities together, and exclude security-review flags. Each issue also carries a severity level, and we count critical issues, those at the two most severe levels (critical and blocker), separately.

We also record the duplicated-code share, the share of lines in duplicated blocks of code. SonarQube also measures code complexity in two ways. Cyclomatic complexity counts the independent paths through the code, and cognitive complexity indexes how difficult its logic is to follow. Table 2 in Appendix A.4 defines each variable.

SonarQube assigns each maintainability issue a standardized repair time, and the sum of these times over a project's code is the project's remediation index (Letouzey, 2012). We convert this index into a dollar value of technical debt following the calibration method from our earlier work (Dimas and Pnevmatikos, 2025, 2026). Our calibration values software at its replacement cost of $27.50 per line of code, about half an hour of development at $55 per hour. We then scale the remediation index so that the sample median technical debt is 20% of the sample median software value (McKinsey & Company, 2020, 2023). We apply this fixed conversion to every sample and date (Appendix A.5).

We express every count of issues, and both complexity measures, per thousand lines of code, excluding blank lines and comments, and measure code size by the same count of lines. Technical debt per line of code, in dollars, measures the maintenance burden of an average line. Because technical debt covers only maintainability issues, we also report total issues per thousand lines, each quality family, duplication, complexity, and code size.

In languages such as Rust, C, and C++, almost none of SonarQube's rules fire, and their quality measures would be near zero by construction (Appendix A.2). We therefore analyze only the nine languages listed in Section 2.2.

Developers usually run SonarQube on the current version of a project as they work. Because our design requires the same measures for past versions, we run it retrospectively. For each repository and month-end, we reconstruct the code from its commit history and scan this *snapshot* (Appendix A.3). We analyze every snapshot in our estimation sample with the same scanner version, SonarQube 25.9, and one fixed set of rules per language. Because past versions of a project often no longer build, the scanner reads source files without compiling them.

2.4 Measuring AI Use

We detect AI use through three channels. The first is *traces* of AI development tools anywhere in a repository's history, such as configuration files for coding agents. The second is code contributions that mention an AI tool as a *coauthor*. The third is *agent pull requests*, opened by one of 23 accounts through which coding agents such as GitHub's Copilot coding agent, Devin, and Google's Jules propose changes. We keep only signals of tools that assist developers and that appear after the tool existed (Appendix A.3). A configuration file or coauthor line typically records an agent working alongside a developer, whereas an agent pull request records a delegated task.

A repository's first detected AI use is the month of its earliest signal in any of the three channels. We treat months outside a channel's coverage as missing, and we exclude a first signal that falls in or before a repository's first covered month, because the use it records may have started earlier. We detect AI use in 2,556 of the 5,566 repositories we scan. Of these, 2,057 show a trace, 1,986 a coauthor mention, and 625 an agent pull request. For 2,220 of them the first signal falls between January 2024 and June 2026, for 65 before 2024, and for 271 after June 2026. Another 3,009 repositories show no signal, and one has no usable record.

The timing of first detection can coincide with other changes in a project, such as rising activity or a new contributor (Renault, Bergeaud, and Bosquet, 2026). We therefore compare *detected projects*, those in which we first detect AI use, with reweighted comparison projects (Section 3.2).

2.5 Estimation Sample and Descriptive Statistics

We index detection cohorts by calendar month c. A project enters cohort c as a detected project if its first signal falls in c and our detection records have covered the project since month $c-6$. We exclude projects that already show a signal when our detection records first cover them. A project enters as a comparison project if our detection records cover it from $c-6$ through at least $c+3$, and its first recorded signal is either absent or later than $c+6$. A project with a signal can therefore serve as a comparison only for cohorts that begin more than six months before its first signal. Later-detected projects are comparison projects whose first signal falls after month $c+6$, and never-detected projects show no signal in our detection records. Our main comparison group consists of later- or never-detected projects, and a second consists of later-detected projects alone.

We consider cohorts first detected from July 2023 through April 2026 and follow each over the twelve months before and the six months after detection, relative months -12 to $+6$, with month -1 as the reference. The estimation sample uses snapshots from October 2022 through July 2026, because the earliest cohort with eligible projects is October 2023. Its key period runs from January 2024 through June 2026, when AI coding tools, and coding agents in particular, spread through software development. This period holds the first detected AI use in 2,220 of the 2,556 projects where we detect it. Our snapshots end in July 2026, so we observe fewer months after detection for the latest cohorts.

We impose four restrictions on the projects in each cohort. First, a project needs at least 5,000 lines of code in month $c - 1$. Second, it needs at least one observed snapshot in months 0 through +6. Third, it needs nearly complete histories over the twelve months before detection, with its code size, total issues, technical debt, cognitive complexity, bugs, and duplicated-code share observed in at least ten of those months. In each six-month half of that period, its commits must be observed in at least three months and its commit authors in at least one. Fourth, we exclude a project from a cohort if its measured code size changes by more than a factor of e (about 2.7) between consecutive observed snapshots in the window. We also drop cohorts with fewer than two comparison projects.

The sample for the main comparison, with later- or never-detected projects, contains 499 detected projects in 27 cohorts, from October 2023 through April 2026. It comprises 42,187 distinct monthly snapshots of 1,459 projects. Across these snapshots we scan 3.92 billion lines of code, and the projects hold 156.65 million lines at their latest snapshot. The sample for the later-detected comparison contains 322 detected projects in 23 cohorts and ends in December 2025. Our detection records end in July 2026, so no project first detected in 2026 can have a comparison project whose AI use we detect after month +6. A project can serve as a comparison in several cohorts, so the stacked regressions contain more observations than distinct snapshots. Appendix Table 3 reports these counts for both comparison groups.

The average detected project holds 144.05 thousand lines of code over the six months before detection. Its code has 22.37 total issues and 1.28 bugs per thousand lines, and $5.571 of technical debt per line. Before weighting, detected projects are larger than later- or never-detected projects in month -1, by 0.61 standard deviations in log code size. They also have fewer total issues and less technical debt per line, by 0.3 and 0.27 standard deviations (Appendix Table 4).

3 Empirical Framework

We estimate how code size and engineering quality change after first detected AI use with a stacked difference-in-differences design. Within each monthly cohort, we reweight comparison projects so that their code size, total issues per line, and technical debt per line follow the same paths as those of detected projects over the prior year.

3.1 Estimation Strategy

Cohort c consists of the projects whose first detected AI use (Section 2.4) falls in calendar month c. We denote by t the month relative to c, with $t = -12, \ldots, +6$ and month -1 as the reference month. We set $A_{ic} = 1$ when project i is a detected project in cohort c and $A_{ic} = 0$ when it serves as one of the cohort's comparison projects.

We compare detected projects with two comparison groups, later- or never-detected projects and later-detected projects, under the sample rules of Section 2.5.

We measure each quality density and share, including complexity per thousand lines, as its change since month -1, and winsorize these changes at the 1st and 99th percentiles within each month t so that a few extreme changes do not drive the estimates.

For code size, the outcome is the natural logarithm of lines, in thousands, measured as its change since month -1. We do not winsorize it, because capping its changes would break the balance that the weights impose on its earlier path. Activity counts and commit shares also enter as changes since month -1, with changes winsorized at the 1st and 99th percentiles within each relative month.

We estimate

$$\widetilde{Y}_{ict} = \alpha_{ic} + \lambda_{c,c+t} + \sum_{\substack{k=-12 \\ k\neq -1}}^{6} \beta_k A_{ic} \mathbf{1}\{t = k\} + \varepsilon_{ict}, \tag{1}$$

where $\widetilde{Y}_{ict}$ is the transformed outcome of project i in cohort c in month t, α_{ic} is a project-by-cohort fixed effect, $\lambda_{c,c+t}$ is a cohort-by-calendar-month fixed effect, $\mathbf{1}\{t = k\}$ indicates month k, and ε_{ict} is the error term. The project-by-cohort effects absorb persistent differences between projects within each cohort panel, and the cohort-by-month effects absorb changes common to all projects of a cohort in a given month. Each β_k measures the weighted difference between detected and comparison projects in month k, relative to month -1.

We estimate equation (1) by weighted least squares. Each detected project receives a weight of one, and the entropy-balancing weights of a cohort's comparison projects (Section 3.2) are scaled to sum to the cohort's number of detected projects. Cohorts therefore enter the estimates approximately in proportion to their number of detected projects. Our main estimate averages the post-detection coefficients. For outcomes in levels, we express the average as a percentage of the pre-detection mean,

$$\widehat{\theta} = \frac{1}{7}\sum_{k=0}^{6} \widehat{\beta}_k, \qquad \widehat{\theta}_{\%} = 100\frac{\widehat{\theta}}{\bar{\mu}}, \qquad \bar{\mu} = \frac{1}{N_A}\sum_{(i,c):A_{ic}=1} \overline{Y}_{ic,[-6,-1]}, \tag{2}$$

where N_A is the number of detected projects and $\overline{Y}_{ic,[-6,-1]}$ is project i's mean outcome, before transformation, over the available months from -6 to -1. The estimate $\widehat{\theta}$ is the average gap over the detection month and the six months that follow, measured from month -1. The pre-detection mean $\bar{\mu}$ only rescales this gap and leaves its reference month unchanged. For log outcomes, including code size, we report $100\widehat{\theta}$ as the approximate percentage-point difference in growth rates relative to comparison projects. We use the same linear scaling for standard errors and confidence intervals.

For each outcome, we test the null hypothesis $H_0\colon \theta = 0$, where $\theta = \frac{1}{7}\sum_{k=0}^{6}\beta_k$, against the

two-sided alternative. We cluster standard errors by repository. First detected AI use is a project-level event, and the same project recurs across cohort panels, as a comparison for several cohorts and, if later detected, in its own cohort. The main regressions for code size, total issues, and technical debt have 1,454 clusters against later- or never-detected projects and 610 against later-detected projects.

We compute the standard error of $\widehat{\theta}$ from the full clustered covariance matrix of the seven monthly coefficients, holding the entropy-balancing weights fixed. Tests of H_0 use the t distribution with $G - 1$ degrees of freedom, where G is the number of clusters. We test whether the coefficients for months -12 through -2 are jointly zero with an F test based on the same clustered covariance matrix.

As a second estimator, we use synthetic difference-in-differences. This estimator replaces the entropy-balancing weights with weights on comparison projects and pre-detection months, which we fit once per cohort to the pre-detection paths of code size, commits, commit authors, and eleven scanner measures, and then hold fixed for every outcome (Appendix B.2).

3.2 Identification Strategy

Our identifying assumption is that, absent first detected AI use, detected projects' outcomes would have moved like those of the reweighted comparison projects. Let $\widetilde{Y}^0_{ict}$ denote this counterfactual transformed outcome of project i in cohort c and month t, and let C_c denote the comparison projects of cohort c. Within each cohort and for every post-detection month $t \geq 0$, we assume

$$\mathbb{E}\big[\widetilde{Y}^0_{ict} \mid A_{ic} = 1\big] = \sum_{j \in C_c} \omega_{jc}\, \mathbb{E}\big[\widetilde{Y}^0_{jct} \mid A_{jc} = 0\big], \tag{3}$$

where the entropy-balancing weights ω_{jc} are nonnegative and sum to one. The later-detected comparison draws the counterfactual only from projects whose AI use we detect after month +6.

First detection is not random in time. Detected projects start larger than later- or never-detected projects, with fewer total issues and less technical debt per line (Section 2.5). We therefore balance the comparison projects on their prior paths of code size and quality. Detection itself requires recorded activity, because an AI trace appears only in a commit, a pull request, or a configuration change, so our sample rules also require observed commits in the year before detection (Section 2.5).

Entropy balancing chooses the weights ω_{jc} within each cohort and comparison group so that the weighted comparison projects match the detected projects on a set of pre-detection moments (Hainmueller, 2012). We first balance the logarithm of code size, total issues per line, and technical debt per line in month -1, together with their eleven monthly changes between months -12 and -1. When no weights satisfy these constraints in a cohort, we balance coarser representations of the same paths and, as a last resort, accept the closest attainable balance (Appendix B.1).

Balancing requires nearly complete pre-detection histories, which the history requirements of Section 2.5 provide. Every outcome and estimator starts from the same estimation sample. We fill the

remaining missing months of each balanced path by linear interpolation, carrying the nearest observed value to the ends, before forming the balancing moments. After weighting, no variable we balance differs between detected and comparison projects by more than 0.02 standard deviations in any month from −12 to −1 against later- or never-detected projects, or by more than 0.03 against later-detected projects. Appendix Table 4 reports the month −1 levels before and after weighting.

For code size, total issues per line, and technical debt per line, the coefficients for months −12 to −2, which measure how closely the reweighted comparison projects track the detected projects after the outcome transformation, are flat against each comparison group (Figures 2, 3, and 4), supporting the identifying assumption.

4 Engineering Quality around First Detected AI Use

Codebases grow faster after first detected AI use, and each line has fewer total issues and fewer bugs. Technical debt per line decreases against later- or never-detected projects ($p = 0.04$), but its decline against later-detected projects is not statistically significant ($p = 0.30$). Security vulnerabilities, security-review flags, critical issues, duplicated-code share, and complexity per line show no statistically significant change in either comparison. Detected projects open more human-authored pull requests, and a larger share of their commits concerns security and fixes. Appendix C reports the estimates behind each figure.

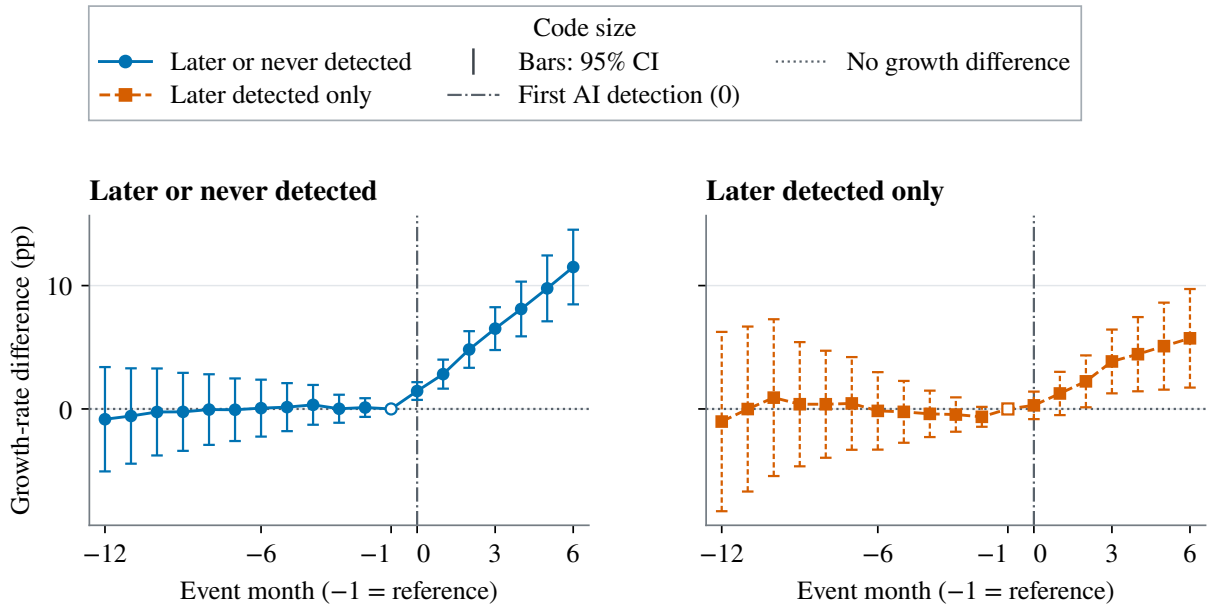
4.1 Codebases Grow Faster after First Detected AI Use

Codebase growth is approximately 6.42 percentage points greater than in projects where we detect later AI usage or none ($p < 0.01$) and 3.26 percentage points greater than in projects where we detect later AI usage ($p < 0.01$), from pre-detection averages of 144.05 and 156.12 thousand lines in the two comparisons (Appendix Table 6). The gap in code size, our measure of how much software a project has built, stays near zero through the year before detection (joint tests $p = 0.99$ and $p = 0.72$) and widens steadily after it (Figure 2).

4.2 Fewer Total Issues per Line

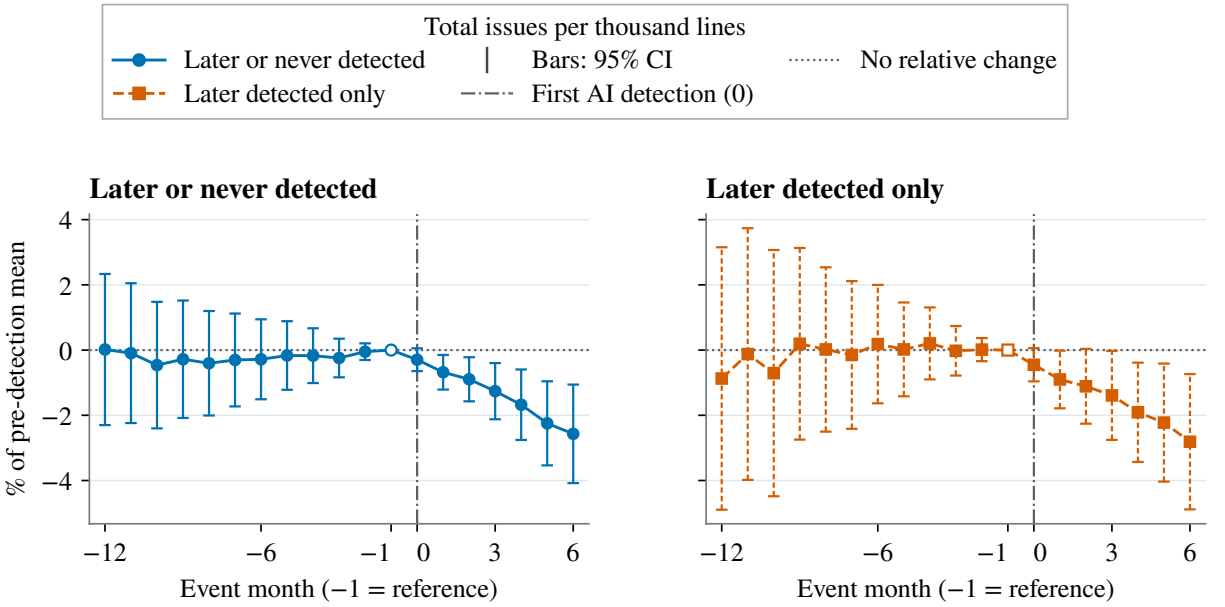
Total issues per line, which combine maintainability issues, bugs, and security vulnerabilities (Section 2.3), decline by 1.37% against later- or never-detected projects ($p < 0.01$) and by 1.54% against later-detected projects ($p = 0.01$). Against later- or never-detected projects, the decline equals 0.3 issues per thousand lines, on a pre-detection mean of 22.37 (Appendix Table 7). After a year of flat coefficients (joint tests $p = 0.97$ and $p = 0.72$), total issues per line decline in every month after detection in each comparison (Figure 3).

Figure 2: Code Size around First Detected AI Use



Notes: Each point is the difference in code size between detected projects and the indicated comparison group, relative to month −1 (month 0 is first detected AI use), and bars show 95% confidence intervals. Points and interval endpoints are 100 times the natural-log code-size estimates, interpreted as approximate percentage-point differences in growth.

Figure 3: Total Issues per Line around First Detected AI Use



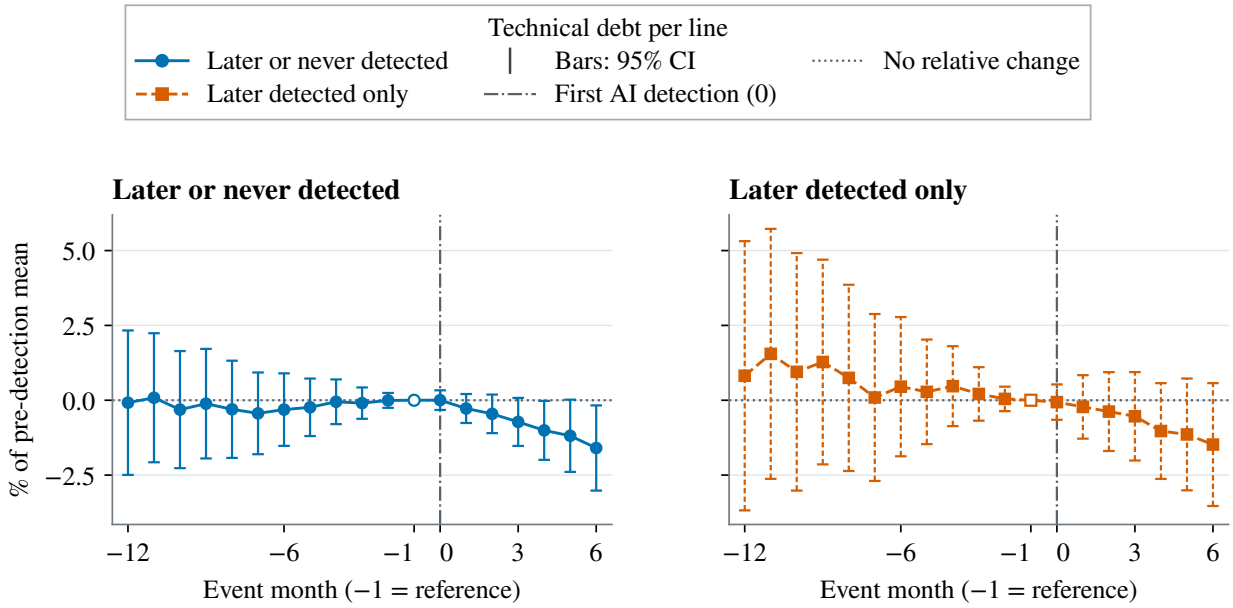
Notes: Each point is the difference in total issues per thousand lines between detected projects and the indicated comparison group, relative to month −1 (month 0 is first detected AI use), and bars show 95% confidence intervals. Responses are percentages of the detected projects' pre-detection mean.

Maintainability issues make up 20.92 of the 22.37 total issues per thousand lines before detection. Maintainability issues decline by a similar percentage (first panel of Figure 7), 1.37% against later- or never-detected projects ($p < 0.01$) and 1.57% against later-detected projects ($p = 0.01$).

4.3 Less Technical Debt per Line

Against later- or never-detected projects, technical debt per line decreases by 0.74% ($p = 0.04$), a decline of \$0.042 per line on a pre-detection mean of \$5.571 per line under the fixed calibration of Section 2.3 (Appendix Table 8). The decline against later-detected projects is about the same size, 0.69%, but it is not statistically significant ($p = 0.30$). The coefficients for technical debt per line stay near zero before detection (joint tests $p = 0.97$ and $p = 0.59$) and turn down after it (Figure 4).

Figure 4: Technical Debt per Line around First Detected AI Use



Notes: Each point is the difference in technical debt in dollars per line between detected projects and the indicated comparison group, relative to month −1 (month 0 is first detected AI use), and bars show 95% confidence intervals. Responses are percentages of the detected projects' pre-detection mean.

4.4 Engineering Quality Relative to Functionality

Functionality provides a proxy for the value firms derive from their software assets. We therefore examine whether the quality improvements documented per line also appear in growth rates per unit of functionality.

We divide total issues and technical debt in dollars by the measure of functionality defined in Section 2.1, and use the natural logarithm of each ratio as the outcome. We retain observations with positive values for both the numerator and denominator.

Relative to comparison projects, growth in total issues per unit of functionality is approximately 6.7 percentage points lower, and growth in technical debt per unit of functionality is approximately 5.0 percentage points lower ($p = 0.03$ and $p = 0.10$, respectively). Figure 5 shows lower growth in both measures after first detected AI use. The estimates cover 188 projects with recorded AI use and their comparison projects.

Comparison projects are weighted to match pre-detection levels and trends in the log ratio, the cumulative feature count, code size, and issues and technical debt per line. Project-by-cohort and cohort-by-calendar-month fixed effects absorb persistent differences and common monthly changes; standard errors are clustered by repository. As in the main results, we summarize months 0 through 6 relative to month -1. The feature-history requirement is at least six observed months for each matching series between months -12 and -4, and a positive observed ratio in month -1 (Appendix E.2).

We also apply the main analysis's matching procedure, matching comparison projects within each month of first detected AI use on prior paths of code size, total issues per line, and technical debt per line. Relaxing the feature-history requirement expands the sample to 225 projects with recorded AI use and their comparison projects. Growth in total issues per unit of functionality is approximately 6.0 percentage points lower ($p = 0.09$). Growth in technical debt per unit of functionality is approximately 4.4 percentage points lower, although not statistically significant ($p = 0.22$). Both measures follow similar downward paths in the two specifications.

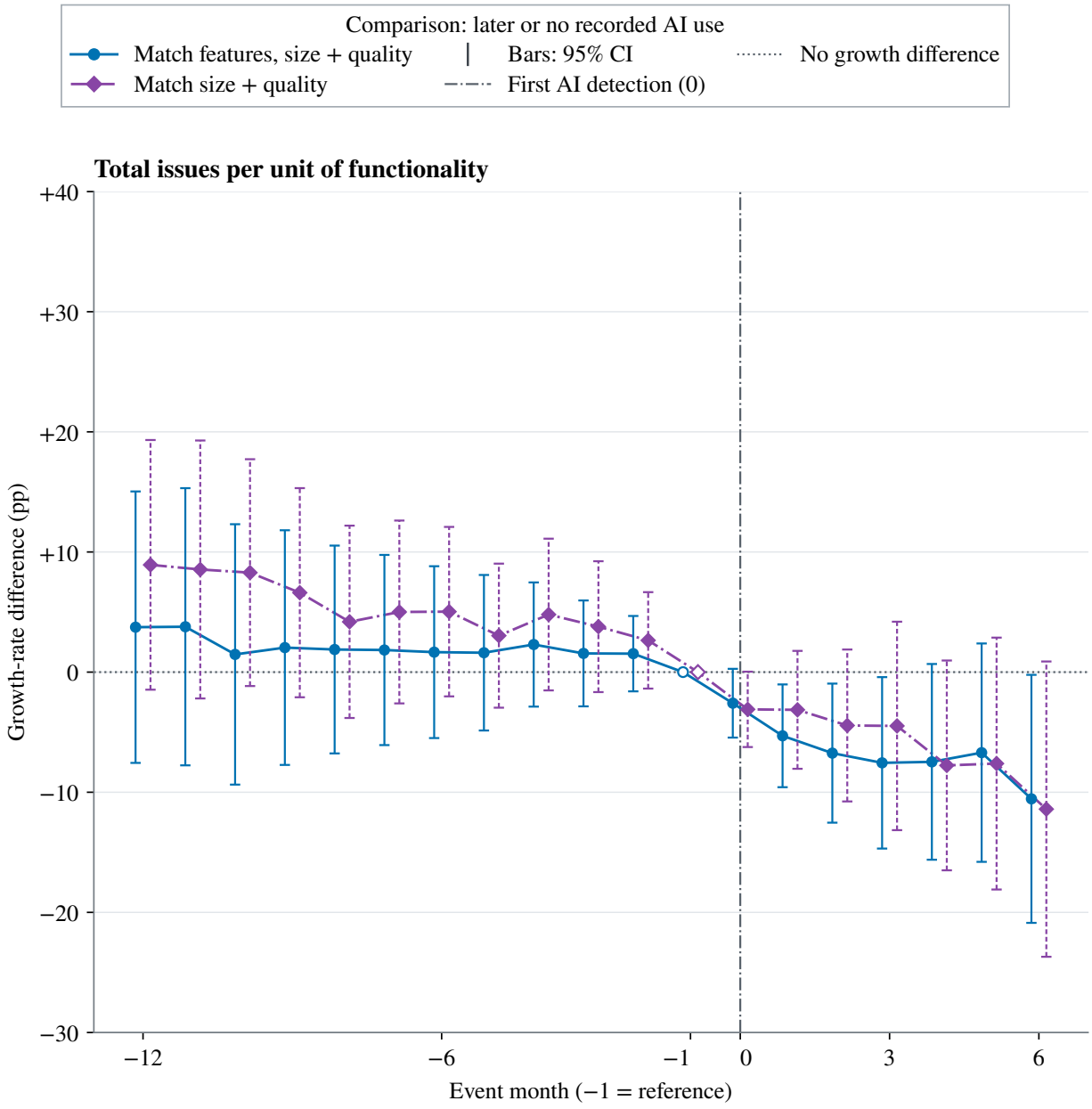
Appendices E.2 and E.3 examine alternative sample restrictions, matching variables, and reference periods. With our main comparison group, the estimates for growth in total issues and technical debt per unit of functionality remain negative across these checks.

4.5 Synthetic Difference-in-Differences and Alternative Outcome Definitions

With synthetic difference-in-differences, our second estimator, we obtain estimates close to the main ones for code size, total issues, bugs, and technical debt (Appendix B). Codebase growth is approximately 6.34 percentage points greater than in projects where we detect later AI usage or none ($p < 0.01$) and 4.89 percentage points greater than in projects where we detect later AI usage ($p < 0.01$). In the same two comparisons, total issues per line decline by 1.09% ($p < 0.01$) and 1.55% ($p = 0.01$). Bugs per line also decline by 1.75% against later- or never-detected projects ($p < 0.01$) and 2.35% against later-detected projects ($p = 0.02$). The synthetic difference-in-differences estimates for technical debt per line are also negative, declines of 0.46% and 0.74%, but neither is statistically significant ($p = 0.26$ and $p = 0.22$).

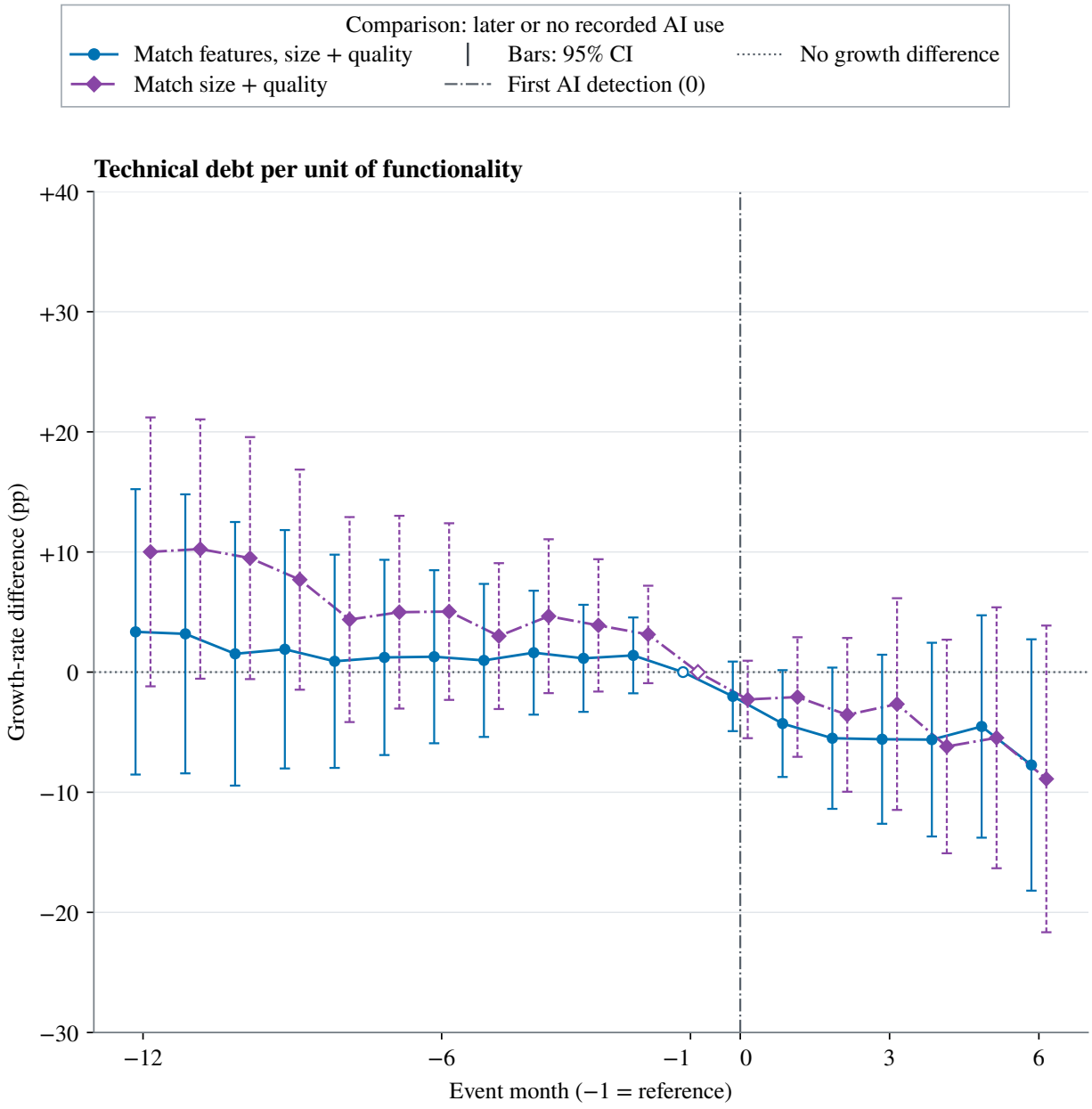
Under two alternative outcome definitions, total issues per line and technical debt per line show

Figure 5: Engineering Quality Relative to Functionality



Notes: Points show $100\beta_k$ and repository-clustered 95% confidence intervals for the log ratio of total issues or technical debt in dollars to cumulative commits labeled as new features, interpreted as approximate percentage-point differences in growth rates; month 0 marks first detected AI use and month −1 is the reference. Blue circles use feature-history matching (188 projects with recorded AI use), while purple diamonds use the broader sample and within-cohort matching on prior code-size and quality paths (225 projects with recorded AI use); both use comparison projects where AI use is recorded later or never.

Figure 5: Engineering Quality Relative to Functionality (continued)



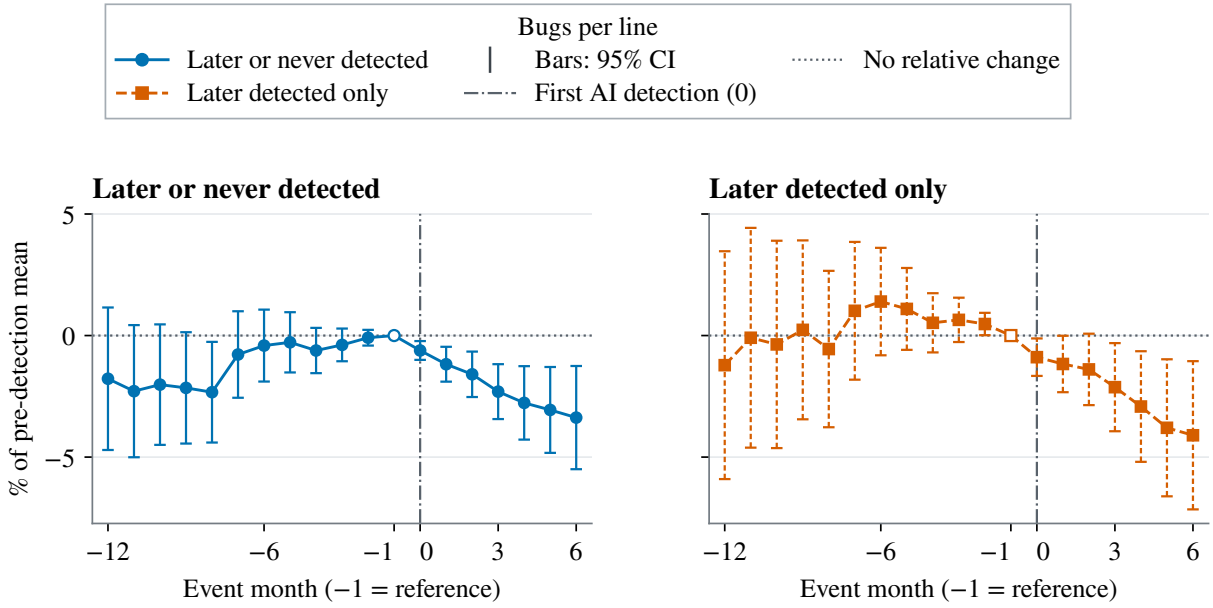
Notes: Points show $100\beta_k$ and repository-clustered 95% confidence intervals for the log ratio of total issues or technical debt in dollars to cumulative commits labeled as new features, interpreted as approximate percentage-point differences in growth rates; month 0 marks first detected AI use and month −1 is the reference. Blue circles use feature-history matching (188 projects with recorded AI use), while purple diamonds use the broader sample and within-cohort matching on prior code-size and quality paths (225 projects with recorded AI use); both use comparison projects where AI use is recorded later or never.

declines of similar or larger size, with larger standard errors. Relative to the average of months −6 to −1 instead of month −1, total issues per line decline by 1.22% against later- or never-detected projects ($p = 0.03$), while the 0.63% decline in technical debt per line is not statistically significant ($p = 0.23$). With density levels in place of winsorized changes, neither the 1.69% decline in total issues per line ($p = 0.10$) nor the 0.94% decline in technical debt per line ($p = 0.23$) is statistically significant (Appendix Table 5).

4.6 Fewer Bugs per Line, No Change in Complexity per Line

Bugs per line decline by 2.12% against later- or never-detected projects ($p < 0.01$) and by 2.34% against later-detected projects ($p < 0.01$), from pre-detection means of 1.28 and 1.27 per thousand lines. The decline builds over the months after detection (Figure 6). Against later- or never-detected projects, the coefficients are negative about a year before detection, rise toward zero by month −2 (joint test $p = 0.03$) and turn down only after detection. Against later-detected projects, the pre-detection coefficients are flat ($p = 0.17$).

Figure 6: Bugs per Line around First Detected AI Use



Notes: Each point is the difference in bugs per thousand lines between detected projects and the indicated comparison group, relative to month −1 (month 0 is first detected AI use), and bars show 95% confidence intervals. Responses are percentages of the detected projects' pre-detection mean.

Security vulnerabilities show no statistically significant change in either comparison ($p = 0.42$ and $p = 0.23$), and the same holds for security-review flags ($p = 0.53$ and $p = 0.47$), critical issues ($p = 0.21$ and $p = 0.89$), and the duplicated-code share ($p = 0.25$ and $p = 0.96$). Figure 7 plots these

outcomes beside maintainability issues. Appendix Table 10 gives each outcome's pre-detection mean and, for security vulnerabilities, the share of detected projects with any vulnerability in month −1.

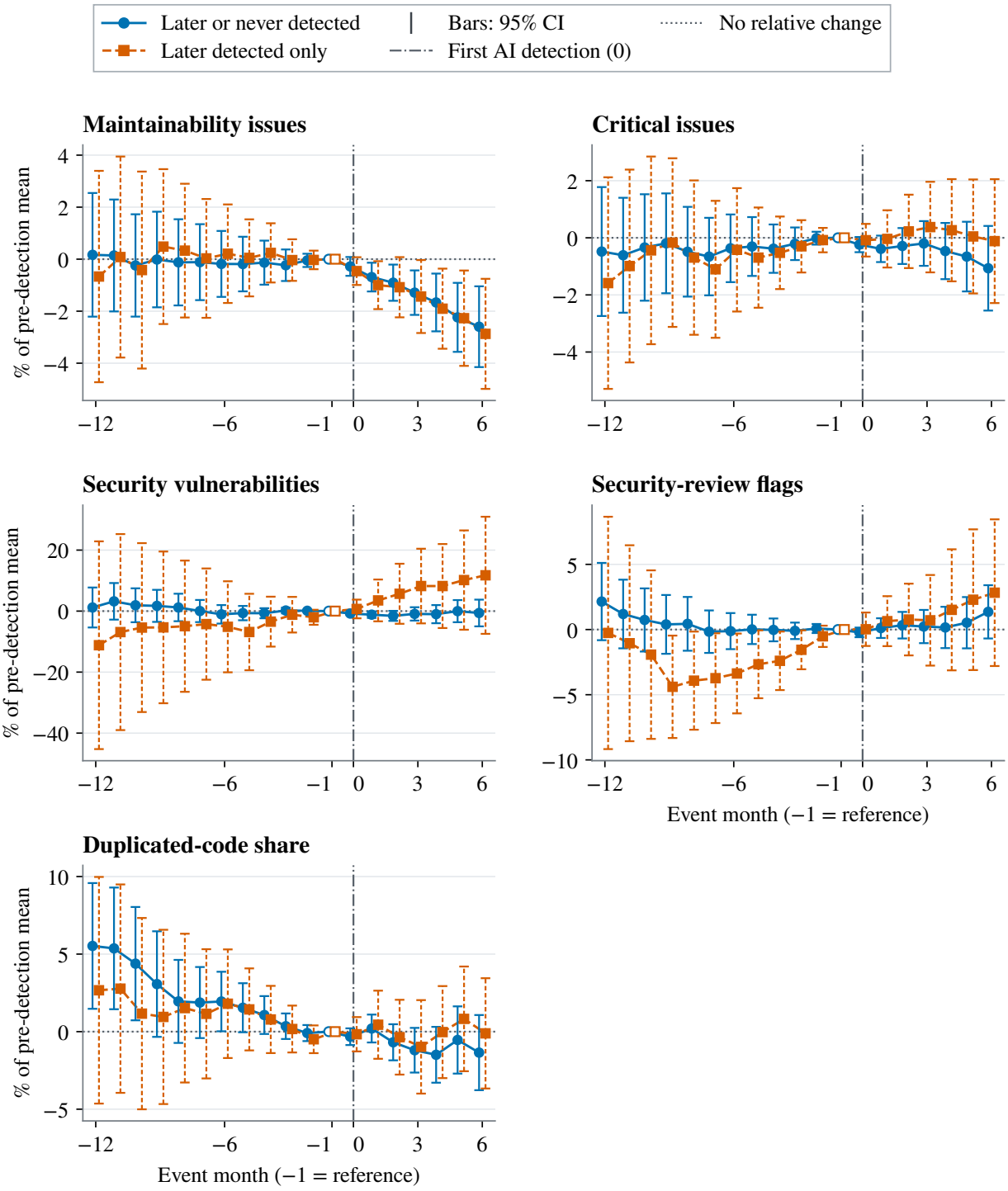
Complexity per line shows no statistically significant change in either comparison, for cognitive complexity ($p = 0.81$ and $p = 0.27$) or for cyclomatic complexity ($p = 0.66$ and $p = 0.88$), the two measures of Section 2.3 (Figure 8 and Appendix Table 10). Before detection, the cognitive-complexity coefficients do not differ jointly from zero in either comparison (joint tests $p = 0.19$ and $p = 0.27$), and neither do the cyclomatic-complexity coefficients ($p = 0.16$ and $p = 0.39$). Total complexity rises as the code grows, and code growth alone predicts most of that rise (Appendix C.1).

4.7 Larger Declines in Technical Debt Where It Starts High

In the top third of pre-detection technical debt per line, technical debt per line decreases by 2.05% against later- or never-detected projects ($p < 0.01$), compared with 0.74% in the full sample. The decline equals \$0.265 per line on a pre-detection mean of \$12.892 per line across 113 detected projects (Appendix Table 9).

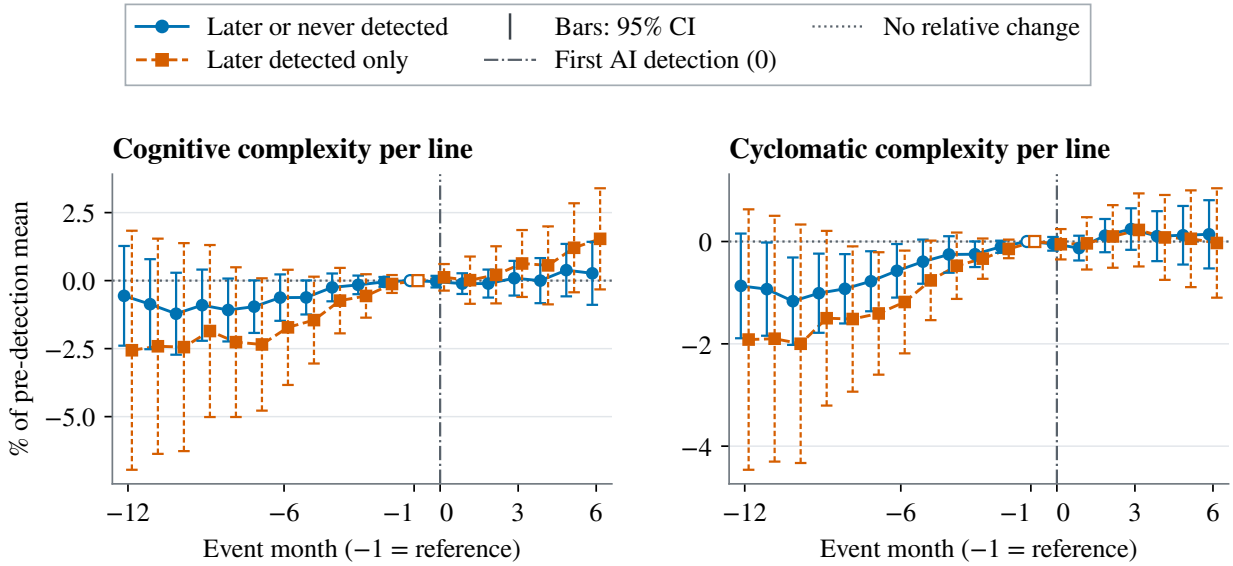
We define this group as the top third of all project–cohort pairs in the sample by mean technical debt per line over months −6 to −1. For each comparison, we keep the detected and comparison projects in this top third and recompute the entropy-balancing weights within this group. The monthly coefficients decline after detection (Figure 9) and are flat before it (joint tests $p = 0.62$ and $p = 0.36$). Against later-detected projects, technical debt per line decreases by 1.68%, but this decline is not statistically significant ($p = 0.29$).

Figure 7: Other Quality Measures around First Detected AI Use



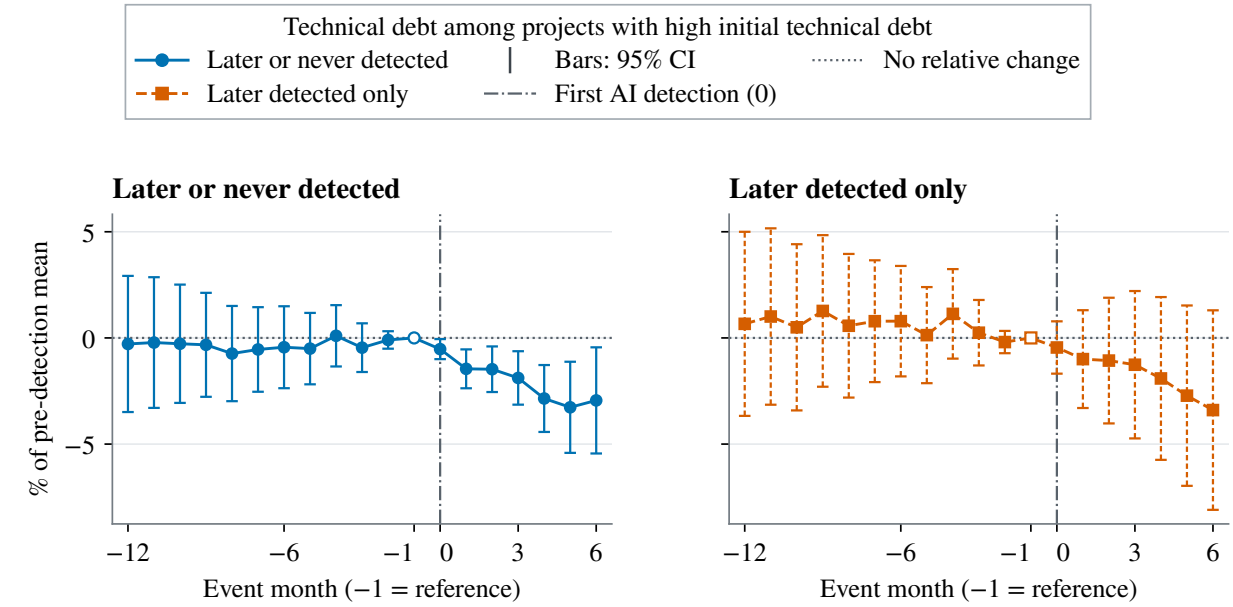
Notes: Each point is the difference in the outcome named in each panel between detected projects and the indicated comparison group, relative to month −1 (month 0 is first detected AI use), and bars show 95% confidence intervals. Responses are percentages of each outcome's pre-detection mean in detected projects.

Figure 8: Complexity per Line around First Detected AI Use



Notes: Each point is the difference in cognitive or cyclomatic complexity per thousand lines between detected projects and the indicated comparison group, relative to month −1 (month 0 is first detected AI use), and bars show 95% confidence intervals. Responses are percentages of each outcome's pre-detection mean in detected projects.

Figure 9: Technical Debt per Line in Projects with High Initial Technical Debt



Notes: Within the top third of pre-detection technical debt per line, each point is the difference in technical debt per line between detected projects and the indicated comparison group, relative to month −1 (month 0 is first detected AI use), and bars show 95% confidence intervals. The comparison weights are recomputed within this group, and responses are percentages of the detected projects' pre-detection mean.

4.8 Quality Gains Concentrate in Projects with Weaker Pre-AI Practices

Quality gains concentrate in projects with weaker development practices before AI use. This pattern points to a source of value beyond faster code production. AI may make it less costly to add checks that a project previously lacked. We examine this possibility using practices that govern how code is reviewed, tested, and integrated.

We summarize five pre-detection practices in a standardized index: human review of pull requests, integration of contributions through pull requests, the share of code files containing tests, the frequency with which code changes also change tests, and configured automatic checks. We divide projects at the detected projects' median index. Within each half, we compare detected projects with later- or never-detected projects, recomputing the entropy-balancing weights on the pre-detection paths of code size, total issues, and technical debt. Appendix D gives the index construction and sample sizes.

Figure 10 shows the difference between the two groups. In the weaker half, total issues per line decline by 2.09% ($p < 0.01$), bugs by 3.04% ($p < 0.01$), and technical debt by 1.84% ($p < 0.01$), relative to comparison projects in the same half. Each response is expressed as a percentage of that half's pre-detection mean. We find no statistically significant quality change in the stronger half. Direct comparisons of these percentage responses show larger quality gains in the weaker half for total issues ($p = 0.02$), bugs ($p = 0.04$), and technical debt ($p < 0.01$), as reported in Appendix Table 15. Codebase growth is approximately 6.85 percentage points greater in the weaker half ($p < 0.01$) and 4.08 percentage points greater in the stronger half ($p < 0.01$), each relative to its comparison group. Projects with weaker initial practices thus combine faster software accumulation with lower measured maintenance needs per line.

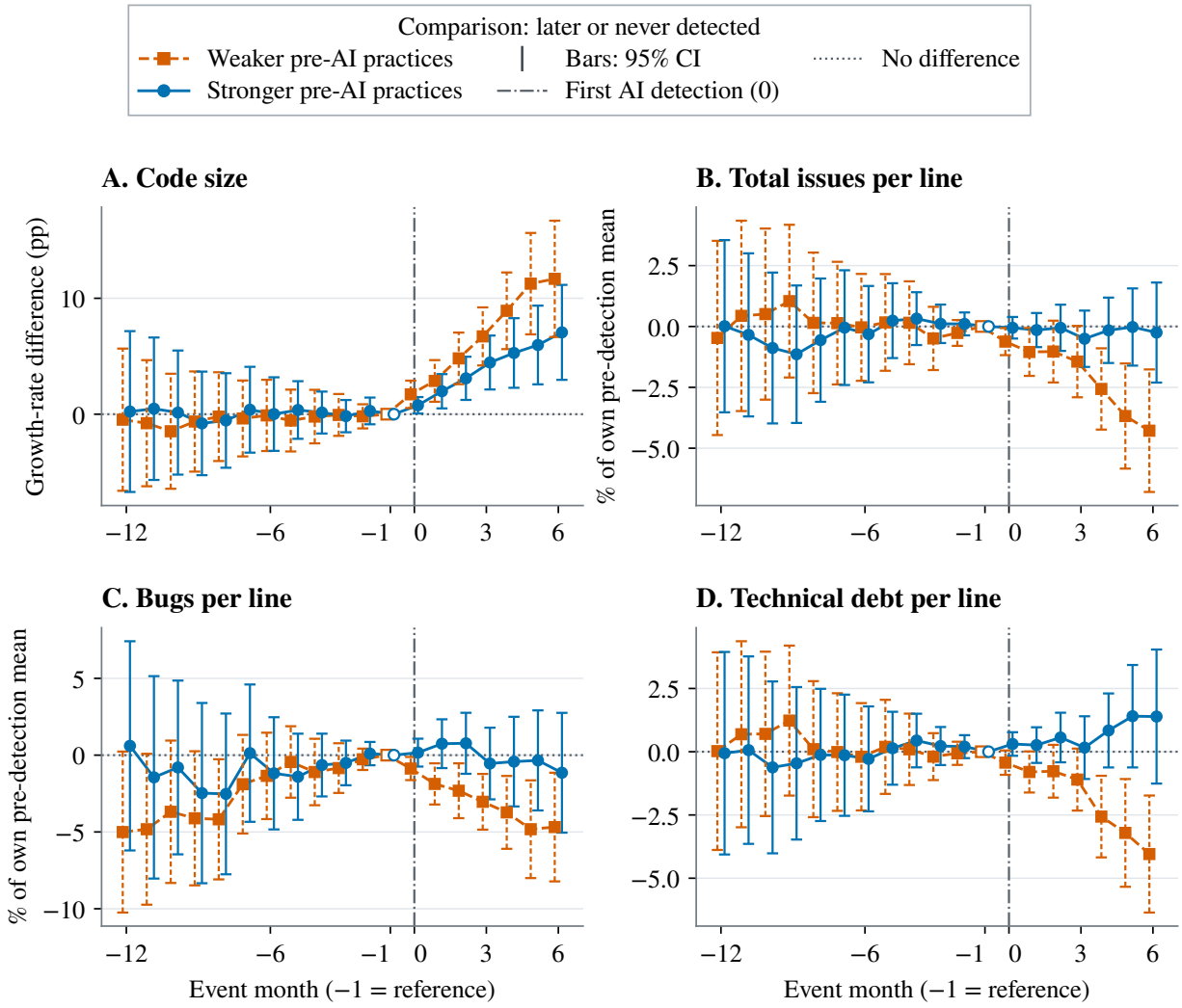
The same ordering appears when comparisons are restricted to later-detected projects, under alternative balancing variables, and within smaller and larger codebases. Splitting the practices separately locates the clearest quality differences in test-file stocks and tests changed with code. Human review points in the same direction for total issues, while proposal integration and configured checks do not distinguish the quality responses. Appendix D reports these comparisons.

Together with the larger decline in projects with high initial technical debt (Section 4.7), these results place the gains where there is more room for improvement. The practice split also suggests how that improvement might occur. Section 4.11 examines whether testing expands where initial checking practices were weaker and whether AI-marked contributions supply those tests.

4.9 More Commits and Pull Requests

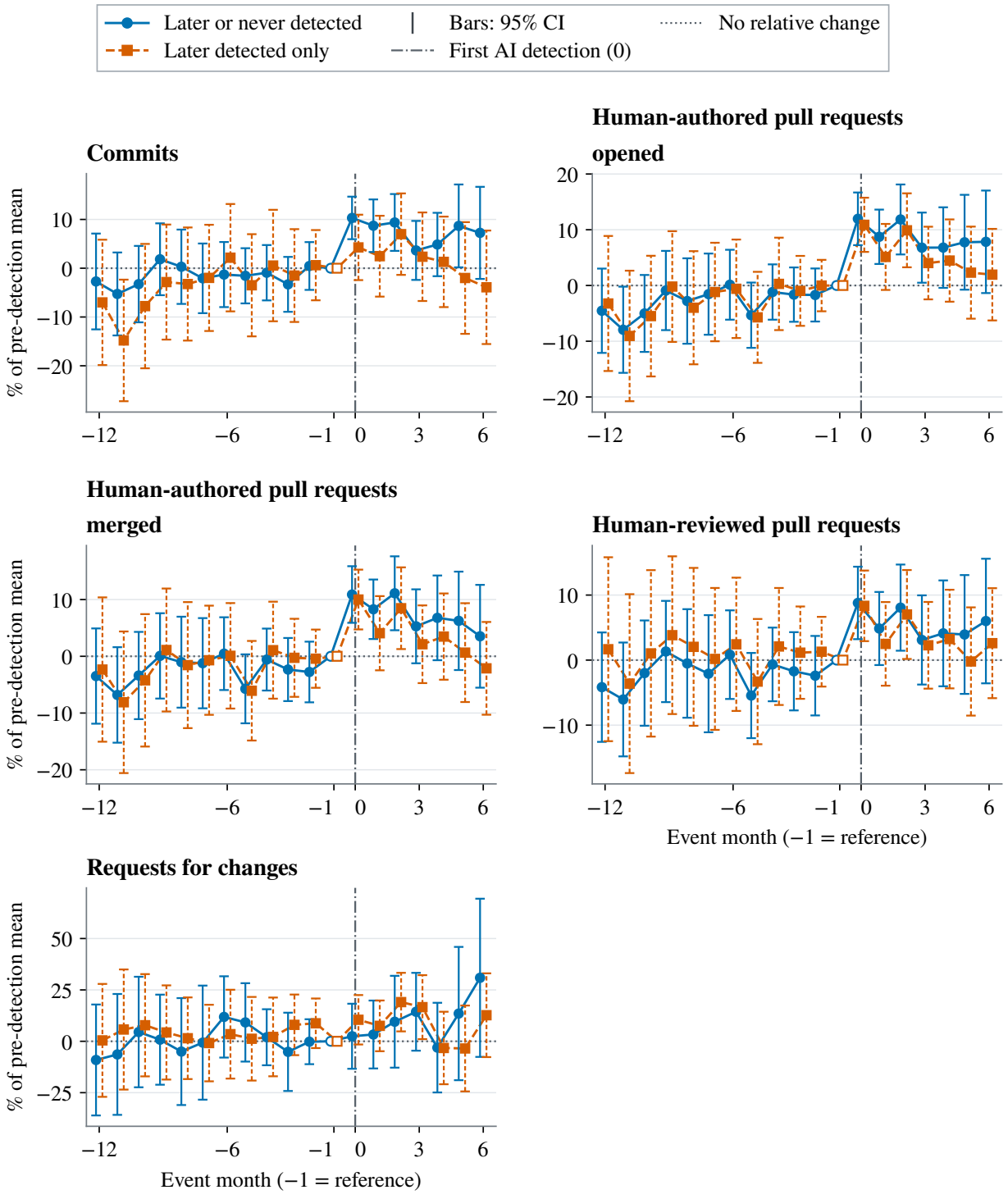
Alongside faster code growth, projects make more commits and open more human-authored pull requests in the detection month, and we see an increase in commits in the first three months (Figure 11 and Appendix Table 11).

Figure 10: Engineering Quality and Code Growth by Pre-AI Discipline



Notes: Later- or never-detected comparisons within the same discipline half; month 0 is first detected AI use and month −1 is the reference. Points show monthly responses with 95% intervals; quality responses use each half's own pre-detection mean, code-size responses are approximate percentage-point differences in growth, and standard errors are clustered by repository.

Figure 11: Development Activity around First Detected AI Use



Notes: Each point is the difference in the monthly count named in each panel between detected projects and the indicated comparison group, relative to month −1 (month 0 is first detected AI use), and bars show 95% confidence intervals. Responses are percentages of each count's pre-detection mean in detected projects, and pull-request panels count human-authored pull requests in the months that our pull-request or review records cover.

Detected projects make 7.54% more commits than later- or never-detected projects ($p < 0.01$), from a pre-detection mean of 54.41 commits a month. The difference from later-detected projects is not statistically significant ($p = 0.64$). The pre-detection commit coefficients are jointly not statistically significant (joint tests $p = 0.55$ and $p = 0.24$).

Detected projects open 8.81% more human-authored pull requests than later- or never-detected projects ($p < 0.01$), from a pre-detection mean of 33.44 a month, and 5.52% more than later-detected projects ($p = 0.03$). The pre-detection pull-request coefficients are jointly not statistically significant (joint tests $p = 0.11$ and $p = 0.12$). Pull requests merged within thirty days rise by 7.44% against later- or never-detected projects ($p < 0.01$), from 27.25 a month, and by a smaller 3.8% against later-detected projects, which is not statistically significant ($p = 0.17$).

Human-reviewed pull requests, those receiving a human review within thirty days, rise by 5.55% against later- or never-detected projects ($p = 0.03$) and by 3.67% against later-detected projects ($p = 0.18$), but only the first estimate is statistically significant. The pre-detection means are 23.93 and 31.87 human-reviewed pull requests a month, respectively. Human requests for changes within thirty days show no statistically significant change against either comparison group ($p = 0.31$ and $p = 0.12$, respectively).

4.10 Larger Shares of Security, Fix, and Contribution-Credit Commits

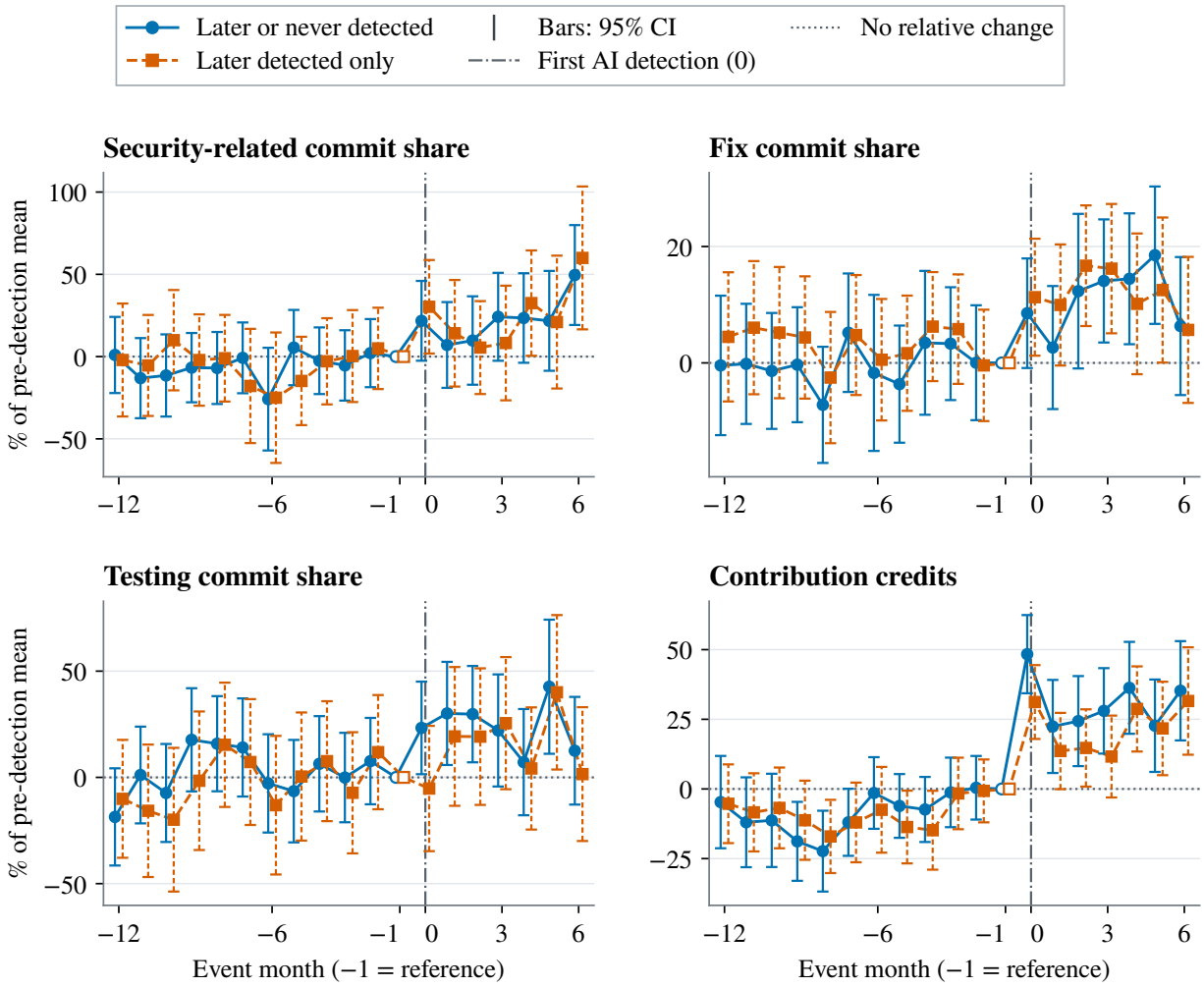
After first detected AI use, the security-related and fix shares of commits climb, and the share of commits with a contribution credit, which can name an AI tool as coauthor (Section 2.1), jumps in the detection month (Figure 12 and Appendix Table 12).

The security-related commit share rises by 22.49% against later- or never-detected projects ($p = 0.02$) and by 24.51% against later-detected projects ($p = 0.04$). On a pre-detection mean of 3.78% of commits, the rise against later- or never-detected projects adds 0.85 percentage points. The fix commit share rises by 10.98% ($p = 0.01$) and 11.78% ($p < 0.01$) in the two comparisons, from pre-detection means of 30.03% and 29.94% of commits. The coefficients of the security-related share are flat before detection (joint tests $p = 0.68$ and $p = 0.93$), and so are those of the fix share ($p = 0.28$ and $p = 0.62$).

The testing commit share rises by 23.94% against later- or never-detected projects ($p = 0.01$), from a pre-detection mean of 4.01% of commits. In this comparison the joint test does not reject flat pre-detection coefficients ($p = 0.10$). The estimate against later-detected projects is smaller, 14.93%, and is not statistically significant ($p = 0.19$).

The share of commits with a contribution credit rises by 31.05% against later- or never-detected projects ($p < 0.01$) and by 21.87% against later-detected projects ($p < 0.01$), from pre-detection means of 13.32% and 14.4% of commits. The joint pre-detection test rejects against later- or never-detected projects ($p = 0.01$) but not against later-detected projects ($p = 0.17$). In detected projects,

Figure 12: Commit Composition around First Detected AI Use



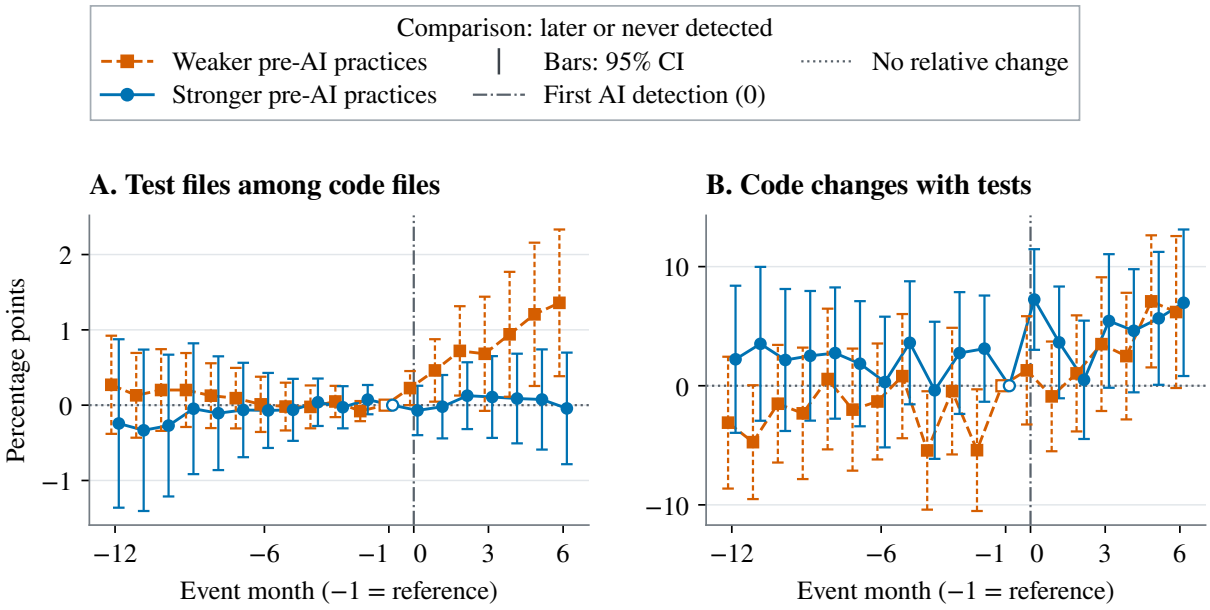
Notes: Each point is the difference in the share of commits named in each panel between detected projects and the indicated comparison group, relative to month −1 (month 0 is first detected AI use), and bars show 95% confidence intervals. Responses are percentages of each share's pre-detection mean in detected projects, and Appendix A.4 defines each share.

contribution credits that name an AI coauthor account for this rise, and those without one do not increase.

4.11 Testing as One Channel of Quality Catch-Up

Tests provide reusable checks on how code should behave as a project evolves. Adding them is an investment in maintaining a growing codebase. In the weaker-practice half, test files rise by 0.79 percentage points as a share of code files ($p < 0.01$; 4.8% of the pre-detection average). The stronger half's change is 0.03 points and is not statistically significant ($p = 0.88$; 0.1% of the pre-detection average). The weaker-half increase also holds against later-detected projects ($p = 0.03$). Figure 13 shows this accumulation of tests. Tests changed alongside code increase in the stronger half ($p = 0.01$); their increase in the weaker half is not statistically significant relative to month -1 ($p = 0.14$). The distinctive catch-up is in the stock of test files.

Figure 13: Testing after First Detected AI Use



Notes: Within each discipline half, detected projects are compared with later- or never-detected projects relative to month −1. Points show unwinsorized changes in percentage points with 95% intervals from repository-clustered standard errors.

AI-marked contributions are one source of these tests. In the weaker half, 48.3% of AI-marked code-changing commits also change a test file, compared with 28.3% of unmarked commits. The difference remains 8.06 percentage points ($p < 0.01$) when we compare contributions within the same project and month and control for commit size. This links the accumulation of tests to the contributions in which developers use AI.

The expansion of testing also helps account for the quality gains. Within a project over time, a

ten-percentage-point increase in test-file share accompanies 4.87% fewer total issues per line, relative to the panel mean, controlling for project and month effects and code size ($p < 0.01$). Adding the change in test-file share to the weaker half's event study reduces the magnitude of its issue coefficient by 19.2% and its technical-debt coefficient by 22.6%, holding the sample fixed. The reduction for bugs is 6%. Table 20 reports the paired estimates. Testing thus accounts for part of the improvement in maintenance needs, while most of the decline in issues and technical debt remains after the adjustment.

The evidence on human review helps distinguish this investment in tests from a broader change in development procedures. We find no statistically significant change in human-review rates, requests for changes, or the fraction of merged proposals with no recorded review. Review counts can still rise as the number of proposals grows, as in Section 4.9. The increase in contribution-credit lines also comes from AI or bot coauthor credits; human reviewer acknowledgments and sign-offs do not rise. Appendix D.3 reports the process estimates. Taken together, the results point to additional checks embedded in the software as one way that AI use can accompany faster growth and lower maintenance needs per line.

4.12 The Results Persist after Excluding Equity-Funded Firms

Investor financing could produce both faster growth and better software if a funding round brings resources for development and maintenance alongside AI adoption (Dimas and Pnevmatikos, 2026). We examine this explanation by excluding projects whose companies have recorded equity funding. We apply the restriction to detected and comparison projects alike and recompute the balancing weights.

Among the 296 detected projects that remain, codebase growth is approximately 5.82 percentage points greater than in comparison projects ($p < 0.01$). Total issues per line decline by 1.54% ($p < 0.01$), bugs by 2.16% ($p < 0.01$), and technical debt by 1.22% ($p < 0.01$), relative to later- or never-detected projects subject to the same restriction. The declines also persist when we restrict the sample to projects linked to companies in the financing data.

Alternative restrictions exclude equity rounds in the event window, ownership changes, or equity funding since 2023. They preserve the same pattern; Appendix D.4 reports the estimates for both comparison groups. Faster software growth, fewer issues, and lower technical debt per line therefore extend beyond firms receiving recorded equity financing around AI adoption.

5 Conclusion

In the corporate public software we study, AI adoption is not linked with a deterioration in the quality of what firms build. We scan the full codebase of each project every month and compare projects in which we first detect AI use with comparable projects whose AI use we detect later or not at all,

and with later-detected projects alone. After first detected AI use, codebases grow faster than those of comparable projects. However, total issues, bugs, and technical debt per line of code decrease modestly relative to these projects.

The decline in total issues per line, most of which are maintainability issues, holds in both comparisons, as does the decline in bugs per line. Technical debt per line decreases by a similar amount in both, although the decline against later-detected projects is not statistically significant. Against later- or never-detected projects, the decline is more pronounced for projects with high initial levels of technical debt. Our main estimates are similar under synthetic difference-in-differences, our second estimator. We find no statistically significant change in the duplicated-code share or complexity per line, which track the engineering shortcuts that AI coding tools can take. We also find no statistically significant change in critical issues, security vulnerabilities, or security-review flags per line. After detection, projects open more human-authored pull requests, which propose changes to the code. A larger share of their code changes concerns security and fixes. Human-reviewed pull requests also rise, although the change against later-detected projects is not statistically significant. Together, these results are not consistent with a story of deterioration in the projects we study.

AI's benefits also depend on the practices already in place. Projects with weaker initial practices show the largest quality gains and add to their stock of tests. AI-marked contributions are particularly likely to include tests. This pattern suggests that AI can help firms invest in the checks needed to maintain a growing software asset.

Quality matters because output that AI makes cheaper to produce may still be costly to maintain, and in most intangible assets, that cost stays hidden until after delivery. In software, we can measure this hidden liability every month, before it appears as rework or failures. Our setting allows us to track total issues, bugs, and security vulnerabilities. Technical debt, the future cost of resolving maintainability issues, puts part of this liability in dollars. Firms founded since the arrival of coding agents may write much of their code with AI, and technical debt can become an obstacle as they scale. Because this liability does not appear in financial statements, investors should measure how much technical debt such a firm carries. Therefore, our methodology could become part of technical due diligence when funding or acquiring a firm. Beyond due diligence, pairing these quality measures with the value of new features and the resources spent on development and maintenance would help firms assess the return on AI investment.

References

Acemoglu, D. 2025. The simple macroeconomics of AI. *Economic Policy* 40:13–58.

Agarwal, S., H. He, and B. Vasilescu. 2026. AI IDEs or autonomous agents? Measuring the impact of coding agents on software development. In *Proceedings of the 23rd International Conference on Mining Software Repositories*. ACM. doi:10.1145/3793302.3793589.

Arkhangelsky, D., S. Athey, D. A. Hirshberg, G. W. Imbens, and S. Wager. 2021. Synthetic difference-in-differences. *American Economic Review* 111:4088–118.

Associated Press. 2025. Deloitte to partially refund Australian government for report with apparent AI-generated errors. October 7.

Babina, T., A. Fedyk, A. He, and J. Hodson. 2024. Artificial intelligence, firm growth, and product innovation. *Journal of Financial Economics* 151:103745.

Banker, R., Y. Liang, and N. Ramasubbu. 2021. Technical debt and firm performance. *Management Science* 67:3174–94.

BetterUp Labs and Stanford Social Media Lab. 2025. Workslop: The hidden cost of AI-generated busywork. Research summary. Survey of 1,150 full-time U.S. desk workers conducted in September 2025. Accessed: October 16, 2026.

Brynjolfsson, E., D. Li, and L. Raymond. 2025. Generative AI at work. *The Quarterly Journal of Economics* 140:889–942.

Conti, A., C. Peukert, and M. Roche. 2025. Beefing IT up for your investor? Engagement with open source communities, innovation, and startup funding: Evidence from GitHub. *Organization Science* 36:1551–73.

Crouzet, N., J. C. Eberly, A. L. Eisfeldt, and D. Papanikolaou. 2022. The economics of intangible capital. *Journal of Economic Perspectives* 36:29–52.

Cui, K. Z., M. Demirer, S. Jaffe, L. Musolff, S. Peng, and T. Salz. 2026. The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers. *Management Science*. Published online February 27, 2026.

Cunningham, W. 1992. The WyCash portfolio management system. In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA '92)*, 29–30. doi:10.1145/157709.157715.

Dell'Acqua, F., E. McFowland, III, E. Mollick, H. Lifshitz, K. C. Kellogg, S. Rajendran, L. Krayer, F. Candelon, and K. R. Lakhani. 2026. Navigating the jagged technological frontier: Field experimental evidence of the effects of artificial intelligence on knowledge worker productivity and quality. *Organization Science* 37:403–23.

Dimas, J., and L. Pnevmatikos. 2025. Cybersecurity vulnerabilities and firm performance. doi:10.2139/ssrn.6520342. Working paper, SSRN 6520342.

———. 2026. Venture capital, engineering quality, and technical debt. doi:10.2139/ssrn.6023094. Working paper, SSRN 6023094.

Eisfeldt, A. L., G. Schubert, and M. B. Zhang. 2023. Generative AI and firm values. NBER Working Paper 31222, National Bureau of Economic Research. doi:10.3386/w31222. Revised January 2026.

Eloundou, T., S. Manning, P. Mishkin, and D. Rock. 2024. GPTs are GPTs: Labor market impact potential of LLMs. *Science* 384:1306–8.

Emery, L. P., C. Lim, and S. Ye. 2025. The private value of open-source innovation. doi:10.2139/ssrn.5054880. Working paper, SSRN 5054880.

Fershtman, C., and N. Gandal. 2011. Direct and indirect knowledge spillovers: The "social network" of open-source projects. *The RAND Journal of Economics* 42:70–91.

Greenstein, S., and F. Nagle. 2014. Digital dark matter and the economic contribution of Apache. *Research Policy* 43:623–31.

Hainmueller, J. 2012. Entropy balancing for causal effects: A multivariate reweighting method to produce balanced samples in observational studies. *Political Analysis* 20:25–46.

Hampole, M., D. Papanikolaou, L. D. W. Schmidt, and B. Seegmiller. 2025. Artificial intelligence and the labor market. NBER Working Paper 33509, National Bureau of Economic Research. doi:10.3386/w33509. Revised September 2025.

He, H., C. Miller, S. Agarwal, C. Kästner, and B. Vasilescu. 2026. Speed at the cost of quality: How Cursor AI increases short-term velocity and long-term complexity in open-source projects. In *Proceedings of the 23rd International Conference on Mining Software Repositories*. ACM. doi:10.1145/3793302.3793349.

Hoffmann, M., S. Boysel, F. Nagle, S. Peng, and K. Xu. 2024. Generative AI and the nature of work. Working Paper 25-021, Harvard Business School. doi:10.2139/ssrn.5007084. Revised April 2025.

Hoffmann, M., F. Nagle, and Y. Zhou. 2024. The value of open source software. Strategy Unit Working Paper 24-038, Harvard Business School. doi:10.2139/ssrn.4693148.

Humlum, A., and E. Vestergaard. 2025. Still waters, rapid currents: Early labor market transformation under generative AI. NBER Working Paper 33777, National Bureau of Economic Research. doi:10.3386/w33777. Revised March 2026. Previously circulated as "Large Language Models, Small Labor Market Effects".

Lerner, J., and J. Tirole. 2002. Some simple economics of open source. *The Journal of Industrial Economics* 50:197–234.

Letouzey, J.-L. 2012. The SQALE method for evaluating technical debt. In *2012 Third International Workshop on Managing Technical Debt (MTD)*, 31–6. Zurich, Switzerland. doi:10.1109/MTD.2012.6225997.

Llanes, G., and R. de Elejalde. 2013. Industry equilibrium with open-source and proprietary firms. *International Journal of Industrial Organization* 31:36–49.

McKinsey & Company. 2020. Tech debt: Reclaiming tech equity. https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/tech-debt-reclaiming-tech-equity. Accessed: March 25, 2025.

———. 2023. Breaking technical debt's vicious cycle to modernize your business. https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/breaking-technical-debts-vicious-cycle-to-modernize-your-business. Accessed: March 25, 2025.

Nagle, F. 2018. Learning by contributing: Gaining competitive advantage through contribution to crowdsourced public goods. *Organization Science* 29:569–87.

———. 2019. Open source software and firm productivity. *Management Science* 65:1191–215.

Noy, S., and W. Zhang. 2023. Experimental evidence on the productivity effects of generative artificial intelligence. *Science* 381:187–92.

Peng, S., E. Kalliamvakou, P. Cihon, and M. Demirer. 2023. The impact of AI on developer productivity: Evidence from GitHub Copilot. arXiv:2302.06590.

Renault, T., A. Bergeaud, and C. Bosquet. 2026. Scientific production in the era of large language models: Outcome-triggered treatment timing and spurious event-study dynamics. *Proceedings of the National Academy of Sciences* 123:e2618638123.

Sarkar, P. 2025. New data on code quality: GPT-5.2 high, Opus 4.5, Gemini 3, and more. Sonar blog. December 15; SonarQube analysis of code generated for more than 4,000 Java programming assignments. Accessed: October 16, 2026.

Stripe, Inc. 2018. The developer coefficient. https://stripe.com/newsroom/stories/developer-coefficient. Accessed: March 25, 2025.

U.S. Bureau of Labor Statistics. 2025. Software developers, quality assurance analysts, and testers. https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm. Median pay: $63.20 per hour (May 2024 data).

U.S. District Court. 2023. Mata v. Avianca, Inc.: Opinion and order on sanctions. Southern District of New York, No. 22-cv-1461 (PKC), Document 54. June 22.

Appendix

A Data Construction and Measurement

This appendix gives the details of the data construction in Section 2.

A.1 Account Discovery and Firm Linkage

The website filter of Table 1 drops accounts whose listed website is a GitHub page or a social-media profile. Accounts that list no website remain at this step. Together with the removal of accounts that no longer exist, this filter leaves 333,248 accounts, which we then match to our repository records by account name. Of the 329,809 linked accounts, 153,877 list a website, which we retrieve and score for signals of commercial activity. Each website receives +1 for each of five company signals (a careers page, a pricing page, an about page, terms of service or a privacy policy, and product pages) and −1 for each signal of a non-commercial site (an open-source community site, a documentation-only site, or a personal page). A website scores −1 if we cannot retrieve it or if its domain belongs to a hosting, blog, or documentation platform. These two cases together account for 27,803 of the 153,877 websites. A net score of at least four therefore requires at least four of the five company signals.

Table 1: Selecting the Repositories We Scan

Step	Repositories	Accounts
Panel A: Discovery of accounts likely to be firms		
Public repositories that at least 50 users have bookmarked	854,735	415,627
Owner account still exists		368,501
Owner's listed website, if any, is not a GitHub or social-media page		333,248
Repositories of these owners	680,613	329,809
Owner lists a website, which is scored		153,877
Website scores at least four on signals of a business		13,511
Repositories of these accounts that at least 50 users have bookmarked	51,915	13,511
Panel B: Filters applied to these accounts' repositories		
Repository still exists on GitHub	49,808	13,086
Not archived by its owner	41,807	12,394
Last push on or after January 1, 2024	26,498	8,389
Among the account's five most-starred remaining repositories	16,441	8,389
Primary language analyzed by the scanner's rules	9,714	5,677
Code the scanner can measure	9,571	5,607
Code as of January 1, 2024 retrieved (repositories whose lines we count)	9,561	5,600
Size screen for at least 5,000 lines (see note)	5,577	3,654

Notes: Each row counts the repositories and owner accounts that remain after the filter in that row, applied in the order shown. A blank cell means the step filters accounts only. Repository counts include repeated entries. The 51,915 candidates contain 92 entries that repeat another repository, and eleven such repeats remain in the final row, which contains 5,566 distinct repositories. Stars count the distinct users who had starred a repository by mid-February 2026. Existence, archiving, last push, and language shares are as recorded between June and October 2026. The size screen uses recorded code size and pre-2024 line counts, with exemptions for newer projects and some projects without an earlier commit; Appendix A.2 gives the rule.

A.2 Filters for the Repositories We Scan

Activity filters. The filters for existence, archiving, and the last push use each repository's status as recorded between June and October 2026. The last-push filter keeps repositories whose most recent push falls on or after January 1, 2024. It excludes projects whose development stopped before 2024. Because the most recent push may come after July 2026, a repository can pass without development during our window. We then keep each account's five most-starred remaining repositories, ranked by the February 2026 star counts that we also use to discover accounts. Applying the activity filters first prevents dormant projects from occupying an account's five places, and the cap keeps a handful of large accounts from dominating the repositories we scan.

Measurability filters. As we configure it, the scanner also reads Rust, C, and C++, which we exclude for the reason given in Section 2.3. The scanner does not analyze languages such as C#, Swift, or Shell. We require that at least half of a repository's current code, measured in bytes, be written in the analyzed languages listed in Section 2.2 or in the web formats that accompany them (HTML, CSS, and XML). We also exclude ten repositories whose code at the start of 2024 we could not retrieve.

Size filter. For repositories created before 2024, we screen for a codebase of at least 5,000 lines at the start of 2024, excluding blank lines and comments. We use each repository's recorded size in 2026: those with less than 60 kilobytes fail and those with more than 500 kilobytes pass. For repositories between these thresholds, including the endpoints, we count lines in the code at the last change before 2024. A repository in this range passes if it has at least 5,000 lines or has no earlier commit. Among 3,467 previously scanned repositories, the 60-kilobyte floor would wrongly remove about 1% of those with at least 5,000 lines in their scans from July 2025 on. For 97% of 300 repositories with a scan from December 2023, the line-counting tool and the scanner place the repository on the same side of 5,000 lines.

Repositories created in 2024 or later, and repositories in the intermediate size range without a commit before 2024, pass the initial size screen. Each project faces the 5,000-line requirement at month $c - 1$ of each detection cohort c in which it enters an estimation sample. Of the 5,577 repositories that pass, 3,152 pass on size in bytes, 983 on a line count, and 1,442 under this exemption (1,372 created in 2024 or later and 70 without an earlier commit).

Duplicate entries. Eleven repositories appear twice among those we scan because, for each of them, two candidate entries resolve to the same project.

A.3 The Scan Collection and Records of AI Use

The scan collection. Each monthly snapshot describes the code present at the end of that month, and a separate timestamp records when we scanned it. We exclude failed scans and observations with missing line counts before constructing outcomes.

Signal filters. For the trace channel, we search the full history of every branch of a repository for configuration files of AI coding agents, commit messages that credit an AI tool, and similar signatures. We keep only traces of tools that assist developers, such as coding agents, editor assistants, and chat tools. Shipping an AI product does not by itself count as AI use. We also discard traces that appear before the corresponding tool existed, and two patterns that hand-labeled validation found to have no true positives. The coauthor channel counts each pattern only from the date the tool could produce it and excludes two human or bot identities whose names merely resemble AI tools.

Signal validation. We recompute the first trace and first agent-pull-request dates from the raw records and compare them with the dates in our detection histories.

Coverage and censoring. A channel observes a repository only in the months its records cover. For the trace channel, coverage runs from the start of the repository's recorded commit history to the month in which we searched the repository for traces. For the coauthor channel, it is the period our commit collection covers, excluding months in which a repository hid previously visible AI signals. For the agent pull-request channel, it is the period our pull-request records cover. Months outside a channel's coverage are missing rather than zero.

Because first detected AI use takes the earliest signal in any channel, a repository is covered in every month that any of its channels covers. We treat a first signal as left-censored, and exclude it, when it falls in or before the repository's first covered month, because the use it records may have started earlier. Our pull-request records cover every month from January 2023, when our detection records begin, so the repositories they include are covered from that month onward.

A.4 Data and Variable Definitions

Table 2 defines the code-quality and development-activity variables and links our outcome names to the scanner and development records. Maintainability issues are the scanner's code smells, and security-review flags are its security hotspots. Total issues sum maintainability issues, bugs, and security vulnerabilities.

Table 2: Data and Variable Definitions

Variable	Description
A. Dimensions of the data	
Project	A public GitHub repository, the archive of a software project's source code and its history of recorded changes. Each project is owned by an account that is likely to be a firm (Section 2.2). We call these projects corporate. An account contributes at most five projects.
Month	Calendar month. We measure each project on a snapshot of its complete codebase at the end of the month. The estimation sample uses snapshots from October 2022 through July 2026.
Detection cohort c	The projects whose first detected AI use falls in calendar month c. Each cohort has its own comparison projects. We consider cohorts first detected from July 2023 through April 2026 and drop cohorts with fewer than two comparison projects.
Unit of observation	A project–cohort–month in the stacked panel. A project can serve as a comparison in several cohorts, so the stacked regressions contain more observations than distinct snapshots.
B. AI detection and comparison groups	
First detected AI use (calendar month)	The month of a project's earliest signal in any of three channels. The first channel is traces of AI development tools anywhere in the project's history, such as configuration files for coding agents. The second is code contributions that mention an AI tool as a coauthor, and the third is agent pull requests, opened by accounts through which coding agents propose changes. We keep only signals of tools that assist developers and that appear after the tool existed. We treat months outside a channel's coverage as missing and exclude a first signal that falls in or before the project's first covered month.
Detected project, A_{ic} (indicator)	Equal to one when project i enters cohort c as a detected project, and zero when it serves as one of the cohort's comparison projects. A project enters as a detected project if its first signal falls in month c and our detection records have covered it since month $c - 6$. We exclude projects that already show a signal when our detection records first cover them.
Comparison project (indicator)	Equal to one for a project that enters cohort c as a comparison. It enters if our detection records cover it from month $c - 6$ through at least $c + 3$, and its first recorded signal is either absent or later than $c + 6$. A project with a signal can therefore serve as a comparison only for cohorts that begin more than six months before its first signal.
Later-detected project (indicator)	Equal to one for a comparison project whose first signal falls after month $c + 6$. Later-detected projects alone form the second comparison group.
Never-detected project (indicator)	Equal to one for a comparison project that shows no signal in our detection records. Later- and never-detected projects together form the main comparison group.
Relative month t (months)	Month relative to the cohort month c, from $t = -12$ to $+6$. Month 0 is the month of first detected AI use, and month -1 is the reference month.
High initial technical debt (indicator)	Equal to one for a project–cohort pair whose mean technical debt per line over months -6 to -1 places it in the top third of all project–cohort pairs in the sample.
C. Engineering quality	
Code size (log thousand lines)	Natural logarithm of lines of code, measured in thousands.
Technical debt per line (USD)	SonarQube's remediation index, the sum of the standardized repair times of a project's maintainability issues, converted to dollars with the fixed calibration and divided by lines of code.
Total issues (per 1,000 lines)	All recorded rule violations per thousand lines of code. The total counts maintainability issues, bugs, and security vulnerabilities, and excludes security-review flags.
Maintainability issues (per 1,000 lines)	Rule violations that make code harder to understand or modify, which SonarQube calls code smells, per thousand lines of code.
Bugs (per 1,000 lines)	Rule violations that can make the software fail or behave unexpectedly when it runs, which SonarQube labels bugs, per thousand lines of code.

Continued on next page

Table 2 (continued)

Variable	Description
Critical issues (per 1,000 lines)	Rule violations at the two most severe levels, critical and blocker, per thousand lines of code.
Security vulnerabilities (per 1,000 lines)	Weaknesses in the code that an attacker could exploit, per thousand lines of code.
Security-review flags (per 1,000 lines)	Flags that SonarQube places on sensitive code for manual review, which it calls security hotspots, per thousand lines of code.
Duplicated-code share (%)	The share of lines in duplicated blocks of code, as SonarQube measures it.
Cognitive complexity per line (per 1,000 lines)	Cognitive complexity, which indexes how difficult the code's logic is to follow, per thousand lines of code.
Cyclomatic complexity per line (per 1,000 lines)	Cyclomatic complexity, which counts the independent paths through the code, per thousand lines of code.
Total cognitive complexity (log)	Natural logarithm of the project's total cognitive complexity, defined when the total is strictly positive.
Total cyclomatic complexity (log)	Natural logarithm of the project's total cyclomatic complexity, defined when the total is strictly positive.
D. Development activity and commit content	
Commits (per month)	Number of commits in the month.
Human-authored pull requests opened and merged (per month)	Human-authored pull requests opened in the month, and those among them merged within thirty days of opening.
Human-reviewed pull requests (per month)	Human-authored pull requests opened in the month that at least one human reviews within thirty days of opening.
Requests for changes (per month)	Human reviews that request changes to a human-authored pull request opened in the month, submitted within thirty days of the pull request's opening.
Security-related and fix commit shares (%)	Commits whose messages match a keyword list for security-related work, or one for fixes, divided by eligible commits in the month (bot commits excluded).
Testing commit share (%)	Commits whose subject lines match a keyword list for testing work, divided by eligible commits in the month (bot commits, merges and release messages excluded).
Contribution credits (%)	Commits whose messages contain a line that credits a reviewer, records a sign-off, or names a coauthor, human or AI, divided by eligible commits in the month (bot commits and merges excluded).

Notes: Engineering-quality measures come from SonarQube Community Edition 25.9, which checks code against thousands of language-specific rules, and every snapshot uses the same scanner version and rules. Lines of code exclude blank lines and comments. Appendix A.5 describes the dollar calibration of technical debt, and the paragraph after this table describes how we classify commits.

Commit classification. A commit counts as security-related or as a fix when its message matches the corresponding keyword list, and as testing work when its subject line matches a list of testing terms. A commit counts toward the contribution-credit share when its message contains a line that credits a reviewer, records a sign-off, or names a coauthor, human or AI. Each share divides the matching commits by the commits eligible for that classification in the repository-month. Commits by bots are never eligible. Merge commits count toward the security and fix shares but not toward the testing or contribution-credit shares, and the testing share also leaves out release messages. A month without eligible commits has a missing share.

A.5 Technical-Debt Calibration

Section 2.3 gives the two inputs of the calibration, a replacement cost of $27.50 per line of code and median technical debt equal to 20% of median software value. They draw on wage data and survey assessments of technical debt (U.S. Bureau of Labor Statistics, 2025; McKinsey & Company, 2023).

The calibration yields one fixed factor that converts the scanner's remediation index into dollars. We apply this factor, and divide by lines of code, before estimation, so the coefficients, standard errors, and pre-detection means in our tables are already in dollars per line of code. We report technical-debt estimates as percentages of the pre-detection mean, and the appendix tables also report them in dollars per line. Percentage estimates and p-values do not depend on the conversion. Dollar levels from calibrations in other samples are not directly comparable with ours.

B Estimation Details and Robustness

B.1 Entropy Balancing and Sample Support

Within each detection cohort c and comparison group, entropy balancing chooses nonnegative weights ω_{jc} for the comparison projects $j \in C_c$ so that their weighted means match the detected projects' means on a set of balancing characteristics (Hainmueller, 2012). The characteristics describe the pre-detection paths of log code size, total issues per thousand lines, and technical debt per line. Each detected project has weight one, and the comparison weights in a cohort initially sum to the number of detected projects.

Path representations and fallback. We describe each balanced path in one of three ways. The monthly representation uses the level in month -1 and the eleven monthly changes between months -12 and -1. The coarse representation uses the same level and the changes from month -12 to -6, from -6 to -3, and from -3 to -1. The twelve-month representation uses the level and the change over the full year. Within a cohort, we try the three in this order, for all three variables at once, and keep the first that the weights can balance exactly. When none can, we apply a ridge-penalized version of entropy balancing, with a penalty of 0.1, to the twelve-month representation.

We first transform the balancing characteristics so that they are uncorrelated and have unit variance in the common sample. The balance criterion requires every weighted comparison mean to lie within 0.02 standard deviations of the detected mean, and we attempt a representation only if the cohort has at least five more comparison projects than balancing characteristics. We then set to zero any comparison weight below 10^{-6} times the largest weight in the cohort.

Sample support and balance. Table 3 reports the common sample and the part of it that receives positive entropy-balancing weights. Table 4 compares the month -1 levels of code size, total issues

per thousand lines, and technical debt per line before and after weighting. Across all twelve monthly levels of these variables, no weighted standardized difference exceeds 0.02 standard deviations against later- or never-detected projects or 0.03 against later-detected projects.

Table 3: Estimation Sample and Weight Support

Common sample	Later or never detected	Later detected
Repositories with AI use	499	322
Comparison repositories	1,368	528
Distinct repositories	1,459	610
Distinct repository–months	42,187	16,706
Detection cohorts	27	23
Entropy-balanced sample		
Repositories with AI use	499	322
Comparison repository–cohorts	15,555	3,953
Comparison repositories	1,359	524
Unique repositories	1,454	610

Notes: The upper block describes the common sample, and the lower block the observations with positive entropy-balancing weights. A repository that serves as a comparison in several cohorts counts once among distinct repositories and repository–months and once per cohort among repository–cohorts.

Table 4: Pre-Detection Balance

	Later or never detected		Later detected	
Characteristic	Before	After	Before	After
Log code size, month −1	0.616	0.001	0.534	0.002
Total issues per thousand lines, month −1	-0.306	0.000	-0.018	0.000
Technical debt per line, month −1	-0.275	-0.001	-0.027	-0.003

Notes: Entries are standardized differences between detected projects and their comparison projects, before and after entropy balancing, in units of the pooled standard deviation. The rows show the month −1 levels of code size, total issues per thousand lines, and technical debt per line, the three variables whose pre-detection paths the weights balance.

B.2 Synthetic Difference-in-Differences

As an alternative estimator, we use synthetic difference-in-differences (Arkhangelsky et al., 2021). For each cohort and comparison group, the estimator chooses one set of weights on comparison projects and one set on pre-detection months in place of the entropy-balancing weights, and uses the same weights for every outcome. We fit these weights jointly to the pre-detection paths of fourteen characteristics: code size, commits, commit authors, technical debt per line, total issues, bugs, security vulnerabilities, security-review flags, maintainability issues, critical issues, cognitive and cyclomatic complexity, and the duplicated-code and comment shares. We measure each characteristic as the transformed outcome $\widetilde{Y}$ of equation (1), divided by its cross-sectional standard deviation over months -12 to -1 within the cohort, so that no characteristic dominates the fit by its units. We include a

project–cohort path if every characteristic is observed in at least seventeen of the nineteen months. We fill the remaining months by linear interpolation, carrying the first and last observed values to the ends, so every included path has all seven post-detection months. A cohort is included if it has at least one detected project and two comparison projects. For technical debt per line, the estimator uses 444 detected projects against later- or never-detected projects and 320 against later-detected projects.

Let C_c be the comparison projects of cohort c and N_c the number of its detected projects. With $\hat{w}_j^c$ the weight on comparison project j and $\hat{v}_t^c$ the weight on pre-detection month t, the estimate is

$$\begin{aligned} \hat{\tau}_c &= \frac{1}{N_c} \sum_{i:A_{ic}=1} \left(\widetilde{Y}_{ic,\text{post}} - \sum_{t=-12}^{-1} \hat{v}_t^c \widetilde{Y}_{ict} \right) - \sum_{j \in C_c} \hat{w}_j^c \left(\widetilde{Y}_{jc,\text{post}} - \sum_{t=-12}^{-1} \hat{v}_t^c \widetilde{Y}_{jct} \right), \\ \hat{\tau} &= \frac{\sum_c N_c \, \hat{\tau}_c}{\sum_c N_c}, \end{aligned} \tag{4}$$

where the subscript post denotes the average over months 0 to +6. The unit weights make the weighted comparison paths track the detected projects' pre-detection paths of all fourteen characteristics at once, each up to its own constant. The time weights make the weighted pre-detection months of each comparison project predict its post-detection mean, again across all fourteen characteristics. Both sets of weights are nonnegative and sum to one, and each carries a ridge penalty. For an outcome outside the fourteen characteristics, we apply the same weights to the included projects whose path of that outcome is observed in at least seventeen of the nineteen months. We express density estimates as a percentage of the pre-detection mean of the detected projects whose paths meet this requirement. We compute this mean before we drop cohorts with too few comparison projects.

We compute standard errors from 100 bootstrap samples. Each sample draws repositories with replacement, keeps all of a drawn repository's cohort appearances, and re-solves the joint weights. Tests use the normal distribution.

B.3 Alternative Baselines and Density Levels

Table 5 reports technical debt per line and total issues per thousand lines under two alternative outcome definitions, next to the main estimates. The first moves the baseline from month -1 to the average of months -6 to -1. We subtract the average of the coefficients $\beta_{-6}, \ldots, \beta_{-1}$, with $\beta_{-1} = 0$, from the average post-detection coefficient. The second uses the density itself as the outcome, without the change since month -1 or winsorization. It keeps the same weights, fixed effects, month -1 reference, and pre-detection mean. The synthetic difference-in-differences estimates appear beside the entropy-balanced estimates in Tables 6–9.

Tests use the repository-clustered covariance and the degrees of freedom described in Section 3, or the repository bootstrap of Appendix B.2 for synthetic difference-in-differences.

Table 5: Alternative Baselines and Density Levels (% of the Pre-Detection Mean)

	Technical debt per line	Total issues per thousand lines
Panel A: Later or never detected		
Winsorized change, reference month −1	-0.748%	-1.375%
	(0.378%)	(0.411%)
Winsorized change, reference months −6 to −1	-0.632%	-1.222%
	(0.528%)	(0.577%)
Density level, reference month −1	-0.943%	-1.696%
	(0.798%)	(1.041%)
# Obs	297,494	297,491
Unique repositories	1,454	1,454
Panel B: Later detected		
Winsorized change, reference month −1	-0.693%	-1.542%
	(0.675%)	(0.630%)
Winsorized change, reference months −6 to −1	-0.936%	-1.609%
	(1.021%)	(0.887%)
Density level, reference month −1	-0.942%	-2.570%
	(1.179%)	(1.514%)
# Obs	80,209	80,207
Unique repositories	610	610
Repository × cohort FE	Yes	Yes
Cohort × calendar-month FE	Yes	Yes

Notes: Columns identify the outcome, and panels identify the comparison group, with entropy-balancing weights throughout. In each panel, the first row repeats the main estimate, the second measures changes from the average of months −6 to −1, and the third uses the density level without winsorization (Appendix B.3). Responses and standard errors are percentages of the detected projects' pre-detection mean. Standard errors cluster by repository.

C Estimates behind the Main Figures

The tables below report the estimates behind the figures of Section 4, together with the pre-detection mean and sample support of each comparison. Each comparison uses its own weights and the observations in which its outcome is measured. Standard errors appear in parentheses below the estimates.

Table 6: Code Size (Natural Log of Lines of Code)

	Later or never detected	Later detected
Panel A: Entropy-balanced event study		
Post-detection response ($100 \times \ln(\text{LoC})$)	6.426	3.270
	(0.837)	(1.225)
# Obs	297,494	80,209
Pre-detection mean (thousand lines)	144.057	156.121
Repositories with AI use	499	322
Comparison repository–cohorts with positive weight	15,555	3,953
Unique repositories	1,454	610
p-value	< 0.01	< 0.01
Panel B: Synthetic difference-in-differences		
Post-detection response ($100 \times \ln(\text{LoC})$)	6.343	4.899
	(0.705)	(1.001)
Project–cohort pairs	16,374	4,632
Pre-detection mean (thousand lines)	147.879	147.879
Repositories with AI use	444	320
Comparison project–cohort pairs	15,930	4,312
Unique comparison repositories	1,326	519
p-value	< 0.01	< 0.01
Repository × cohort FE (Panel A)	Yes	Yes
Cohort × calendar-month FE (Panel A)	Yes	Yes

Notes: Columns identify the comparison group, and panels identify the estimator. The dependent variable is 100 times the natural log of lines of code, and pre-detection means are in thousand lines. Panel A averages months 0 to +6 relative to month −1, and Panel B compares the same months with time-weighted pre-detection months. Project–cohort pairs count each repository separately in each cohort-specific fit. In Panel B, each pair contributes a 19-month estimation window, including interpolated values where required. Standard errors cluster by repository in Panel A and come from 100 repository bootstrap samples in Panel B.

Table 7: Total Issues per Thousand Lines

	Later or never detected	Later detected
Panel A: Entropy-balanced event study		
Post-detection response	-0.307	-0.345
	(0.092)	(0.141)
# Obs	297,491	80,207
Response / pre-detection mean	-1.375%	-1.542%
Pre-detection mean (per thousand lines)	22.370	22.377
Repositories with AI use	499	322
Comparison repository–cohorts with positive weight	15,555	3,953
Unique repositories	1,454	610
p-value	< 0.01	0.01
Panel B: Synthetic difference-in-differences		
Post-detection response	-0.241	-0.344
	(0.092)	(0.140)
Project–cohort pairs	16,374	4,632
Response / pre-detection mean	-1.091%	-1.558%
Pre-detection mean (per thousand lines)	22.088	22.088
Repositories with AI use	444	320
Comparison project–cohort pairs	15,930	4,312
Unique comparison repositories	1,326	519
p-value	< 0.01	0.01
Repository × cohort FE (Panel A)	Yes	Yes
Cohort × calendar-month FE (Panel A)	Yes	Yes

Notes: Columns identify the comparison group, and panels identify the estimator. The dependent variable counts total issues per thousand lines of code, and the percentage row divides each response by the detected projects' pre-detection mean. Panel A averages months 0 to +6 relative to month −1, and Panel B compares the same months with time-weighted pre-detection months. Project–cohort pairs count each repository separately in each cohort-specific fit. In Panel B, each pair contributes a 19-month estimation window, including interpolated values where required. Standard errors cluster by repository in Panel A and come from 100 repository bootstrap samples in Panel B.

Table 8: Technical Debt per Line (Dollars per Line of Code)

	Later or never detected	Later detected
Panel A: Entropy-balanced event study		
Post-detection response	-0.042	-0.039
	(0.021)	(0.038)
# Obs	297,494	80,209
Response / pre-detection mean	-0.748%	-0.693%
Pre-detection mean ($ per LoC)	5.571	5.604
Repositories with AI use	499	322
Comparison repository–cohorts with positive weight	15,555	3,953
Unique repositories	1,454	610
p-value	0.04	0.30
Panel B: Synthetic difference-in-differences		
Post-detection response	-0.026	-0.042
	(0.023)	(0.034)
Project–cohort pairs	16,374	4,632
Response / pre-detection mean	-0.464%	-0.748%
Pre-detection mean ($ per LoC)	5.569	5.569
Repositories with AI use	444	320
Comparison project–cohort pairs	15,930	4,312
Unique comparison repositories	1,326	519
p-value	0.26	0.22
Repository × cohort FE (Panel A)	Yes	Yes
Cohort × calendar-month FE (Panel A)	Yes	Yes

Notes: Columns identify the comparison group, and panels identify the estimator. The dependent variable is technical debt in dollars per line of code under the fixed calibration of Appendix A.5, and the percentage row divides each response by the detected projects' pre-detection mean. Panel A averages months 0 to +6 relative to month −1, and Panel B compares the same months with time-weighted pre-detection months. Project–cohort pairs count each repository separately in each cohort-specific fit. In Panel B, each pair contributes a 19-month estimation window, including interpolated values where required. Standard errors cluster by repository in Panel A and come from 100 repository bootstrap samples in Panel B.

Table 9: Technical Debt per Line in Projects with High Initial Technical Debt (Dollars per Line of Code)

	Later or never detected	Later detected
Panel A: Entropy-balanced event study		
Post-detection response	-0.265	-0.225
	(0.082)	(0.215)
# Obs	79,663	14,242
Response / pre-detection mean	-2.058%	-1.686%
Pre-detection mean ($ per LoC)	12.892	13.354
Repositories with AI use	113	71
Comparison repository–cohorts with positive weight	4,155	680
p-value	< 0.01	0.29
Panel B: Synthetic difference-in-differences		
Post-detection response	-0.229	-0.303
	(0.082)	(0.171)
Project–cohort pairs	4,489	769
Response / pre-detection mean	-1.755%	-2.322%
Pre-detection mean ($ per LoC)	13.030	13.030
Repositories with AI use	99	71
Comparison project–cohort pairs	4,390	698
Unique comparison repositories	414	115
p-value	< 0.01	0.07
Repository × cohort FE (Panel A)	Yes	Yes
Cohort × calendar-month FE (Panel A)	Yes	Yes

Notes: Columns identify the comparison group, and panels identify the estimator. The sample keeps detected and comparison projects in the top third of pre-detection technical debt per line, and we re-solve the entropy-balancing weights within it. The dependent variable is technical debt in dollars per line of code. Panel A averages months 0 to +6 relative to month −1, and Panel B compares the same months with time-weighted pre-detection months. Project–cohort pairs count each repository separately in each cohort-specific fit. In Panel B, each pair contributes a 19-month estimation window, including interpolated values where required. Standard errors cluster by repository in Panel A and come from 100 repository bootstrap samples in Panel B.

Table 10: Other Engineering-Quality Outcomes (% of the Pre-Detection Mean)

Outcome	Later or never detected	Later detected
Maintainability issues	-1.380%	-1.574%
	(0.421%)	(0.642%)
# Obs	297,494	80,209
Pre-detection mean (per thousand lines)	20.920	21.036
Repositories with AI use / unique repositories	499 / 1,454	322 / 610
p-value	< 0.01	0.01
Bugs	-2.129%	-2.342%
	(0.561%)	(0.895%)
# Obs	297,494	80,209
Pre-detection mean (per thousand lines)	1.289	1.279
Repositories with AI use / unique repositories	499 / 1,454	322 / 610
p-value	< 0.01	< 0.01
Critical issues	-0.471%	0.096%
	(0.379%)	(0.719%)
# Obs	297,494	80,209
Pre-detection mean (per thousand lines)	7.513	7.338
Repositories with AI use / unique repositories	499 / 1,454	322 / 610
p-value	0.21	0.89
Security vulnerabilities	-0.871%	6.889%
	(1.091%)	(5.854%)
# Obs	297,315	80,200
Pre-detection mean (per thousand lines)	0.071	0.064
Repositories with AI use / unique repositories	499 / 1,454	322 / 610
p-value	0.42	0.23
Nonzero at month -1	47.295%	50.932%
Security-review flags	0.356%	1.252%
	(0.576%)	(1.761%)
# Obs	297,494	80,209
Pre-detection mean (per thousand lines)	1.320	1.126
Repositories with AI use / unique repositories	499 / 1,454	322 / 610
p-value	0.53	0.47
Duplicated-code share	-0.770%	-0.054%
	(0.680%)	(1.258%)
# Obs	297,494	80,209
Pre-detection mean	9.786%	9.660%
Repositories with AI use / unique repositories	499 / 1,454	322 / 610
p-value	0.25	0.96
Cognitive complexity per line	0.069%	0.612%
	(0.300%)	(0.562%)
# Obs	297,494	80,209
Pre-detection mean (per thousand lines)	108.826	109.187
Repositories with AI use / unique repositories	499 / 1,454	322 / 610
p-value	0.81	0.27
Cyclomatic complexity per line	0.079%	0.049%
	(0.183%)	(0.330%)
# Obs	297,494	80,209
Pre-detection mean (per thousand lines)	128.217	127.843
Repositories with AI use / unique repositories	499 / 1,454	322 / 610
p-value	0.66	0.88
Repository × cohort FE	Yes	Yes
Cohort × calendar-month FE	Yes	Yes

Notes: Rows identify outcomes, and columns identify the comparison group. Each issue density counts the issues named in its row per thousand lines of code, complexity per line counts complexity points per thousand lines, and duplicated-code share is a percentage of lines. Responses and standard errors are percentages of each outcome's pre-detection mean, which the table reports in native units, and the vulnerability row also reports the share of detected projects with any vulnerability in month -1. Standard errors cluster by repository.

Table 11: Development Activity and Human Review (% of the Pre-Detection Mean)

Outcome	Later or never detected	Later detected
Commits	7.548%	1.637%
	(2.438%)	(3.579%)
# Obs	297,081	79,914
Pre-detection mean (per month)	54.414	66.160
Repositories with AI use / unique repositories	499 / 1,454	322 / 610
p-value	< 0.01	0.64
Human-authored pull requests opened	8.816%	5.521%
	(2.535%)	(2.649%)
# Obs	167,053	72,045
Pre-detection mean (per month)	33.449	43.933
Repositories with AI use / unique repositories	462 / 906	303 / 554
p-value	< 0.01	0.03
Human-authored pull requests merged	7.444%	3.809%
	(2.542%)	(2.774%)
# Obs	167,053	72,045
Pre-detection mean (per month)	27.253	35.788
Repositories with AI use / unique repositories	462 / 906	303 / 554
p-value	< 0.01	0.17
Human-reviewed pull requests	5.554%	3.679%
	(2.667%)	(2.753%)
# Obs	161,889	71,609
Pre-detection mean (per month)	23.938	31.880
Repositories with AI use / unique repositories	460 / 894	302 / 551
p-value	0.03	0.18
Requests for changes	10.134%	8.513%
	(9.985%)	(5.539%)
# Obs	161,889	71,609
Pre-detection mean (per month)	2.212	2.842
Repositories with AI use / unique repositories	460 / 894	302 / 551
p-value	0.31	0.12
Repository × cohort FE	Yes	Yes
Cohort × calendar-month FE	Yes	Yes

Notes: Rows identify outcomes, and columns identify the comparison group. The outcomes are monthly commits, human-authored pull requests opened, those merged or receiving a human review within thirty days of opening, and human requests for changes within that window. Responses and standard errors are percentages of each outcome's pre-detection mean, which the table reports per month. Standard errors cluster by repository.

Table 12: Commit Composition (% of the Pre-Detection Mean)

Outcome	Later or never detected	Later detected
Security-related commit share	22.497%	24.520%
	(9.865%)	(12.160%)
# Obs	125,678	59,338
Pre-detection mean	3.785%	3.418%
Repositories with AI use / unique repositories	423 / 1,201	286 / 578
p-value	0.02	0.04
Fix commit share	10.982%	11.782%
	(4.410%)	(4.471%)
# Obs	125,678	59,338
Pre-detection mean	30.032%	29.946%
Repositories with AI use / unique repositories	423 / 1,201	286 / 578
p-value	0.01	< 0.01
Testing commit share	23.944%	14.932%
	(9.315%)	(11.633%)
# Obs	123,807	58,982
Pre-detection mean	4.013%	3.659%
Repositories with AI use / unique repositories	423 / 1,199	287 / 578
p-value	0.01	0.19
Contribution credits	31.057%	21.877%
	(6.210%)	(5.811%)
# Obs	124,374	59,131
Pre-detection mean	13.328%	14.406%
Repositories with AI use / unique repositories	424 / 1,200	287 / 578
p-value	< 0.01	< 0.01
Repository × cohort FE	Yes	Yes
Cohort × calendar-month FE	Yes	Yes

Notes: Rows identify outcomes, and columns identify the comparison group. Each outcome is the share of eligible commits that a keyword list flags as security-related, fix, or testing work, or that contain a contribution credit, including AI coauthor lines (Appendix A.4). Months without eligible commits are missing. Responses and standard errors are percentages of each share's pre-detection mean, which the table reports as a percentage of commits. Standard errors cluster by repository.

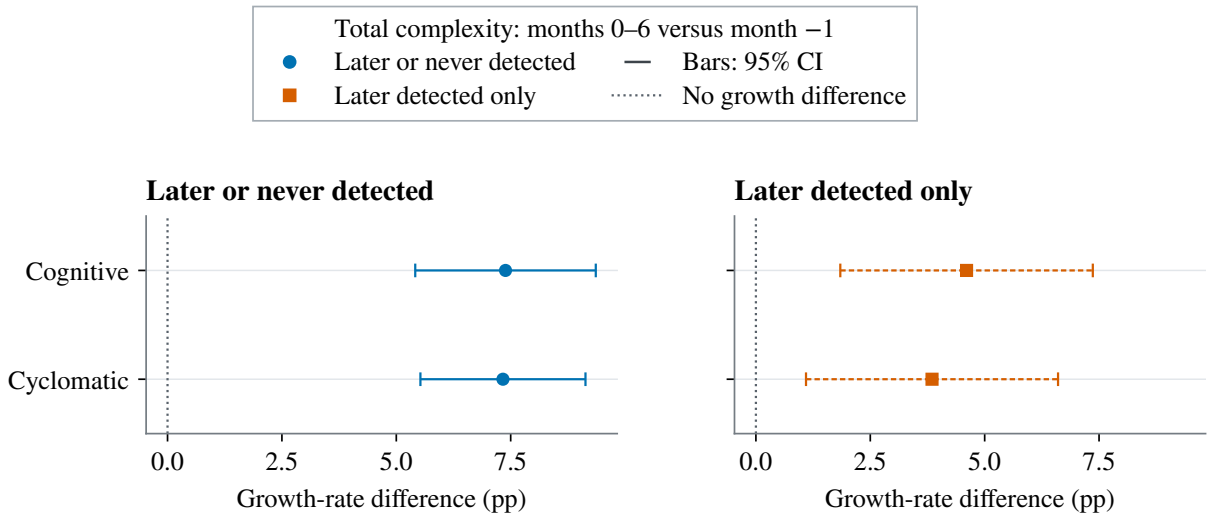
C.1 Total Complexity and Code Growth

After first detected AI use, growth in total cognitive complexity is approximately 7.38 percentage points greater than in projects where we detect later AI usage or none ($p < 0.01$) and 4.6 percentage points greater than in projects where we detect later AI usage ($p < 0.01$). Growth in total cyclomatic complexity is approximately 7.33 percentage points greater ($p < 0.01$) and 3.84 percentage points greater ($p < 0.01$) in the same two comparisons. Before detection, the coefficients do not differ jointly from zero for total cognitive complexity (joint tests $p = 0.95$ and $p = 0.22$) or for total cyclomatic complexity ($p = 0.84$ and $p = 0.42$). Figure 14 and Table 13 show these estimates, which use the natural logarithm of each project's total as the outcome and leave out months with a zero total. We report $100\widehat{\theta}$ as an approximate percentage-point difference in growth rates.

Code growth alone predicts most of this rise. Before first detected AI use, total complexity rises almost one for one with lines of code within a project, with elasticities of 0.96 for cognitive and 0.91 for cyclomatic complexity.[1] At these elasticities, the approximately 6.42 percentage-point difference in codebase growth against projects where we detect later AI usage or none predicts growth differences of approximately 6.18 percentage points in total cognitive and 5.85 percentage points in total cyclomatic complexity. Measured as changes in natural logs, these predicted rises equal 83.7% and 79.9% of the estimated ones. Section 4.6 therefore reports complexity per thousand lines, which normalizes complexity by code size like every other quality measure.

[1]The regression of the natural log of each total on the natural log of lines of code includes project and calendar-month fixed effects, clusters standard errors by repository, and uses snapshots with strictly positive values taken before the project's first detected AI use, or before 2024 for never-detected projects.

Figure 14: Average Change in Total Code Complexity after First Detected AI Use



Notes: Points show the average change in each total over months 0 to +6 relative to month −1, against the indicated comparison group, and bars show 95% confidence intervals. Points and interval endpoints are 100 times the natural-log estimates, interpreted as approximate percentage-point differences in growth; the estimates use strictly positive totals.

Table 13: Total Cognitive and Cyclomatic Complexity (Natural Log of Repository Totals)

	Cognitive complexity	Cyclomatic complexity
Panel A: Later or never detected		
Post-detection response (100 × ln(total))	7.387	7.331
	(1.006)	(0.920)
# Obs	297,383	297,494
Pre-detection mean, positive totals	15,324.717	18,326.664
Repositories with AI use	499	499
Unique repositories	1,453	1,454
p-value	< 0.01	< 0.01
Panel B: Later detected		
Post-detection response (100 × ln(total))	4.604	3.849
	(1.405)	(1.402)
# Obs	80,209	80,209
Pre-detection mean, positive totals	16,067.077	19,314.985
Repositories with AI use	322	322
Unique repositories	610	610
p-value	< 0.01	< 0.01
Repository × cohort FE	Yes	Yes
Cohort × calendar-month FE	Yes	Yes

Notes: Columns identify the complexity measure, and panels identify the comparison group. Each dependent variable is 100 times the natural log of the repository total, among repositories with a positive total, and the reported response is an approximate percentage-point difference in growth rates. Pre-detection means are complexity points in the same repositories. The response averages months 0 to +6 relative to month −1. Standard errors cluster by repository.

D Pre-AI Development Practices, Testing, and Financing

D.1 Measuring Practices before First Detected AI Use

The index combines five practices observed before first detected AI use. We construct each measure for a project–cohort observation, using month -1 for stocks and months -12 to -1 for flows. Human review is the share of human-authored pull requests receiving a human review within thirty days of opening and before the start of month 0. We aggregate proposals opened in covered months -12 to -1 and require at least six proposals. A first-parent history follows the main line of development when branches are combined. Proposal integration is the share of non-bot first-parent commits integrated through a pull request over the same months, requiring at least twelve commits. A squash commit combines a proposal's changes into a single commit. We identify integration from merge messages and the proposal identifiers in squash commit messages.

Test-file share is the fraction of programming-language files classified as tests in the file tree at month -1, requiring at least ten code files and excluding vendored directories, which contain copies of third-party code. Tests changed with code is the share of non-bot, code-changing first-parent commits that also change a test file over months -12 to -1, requiring at least twelve code-changing commits. Finally, we count the presence of four types of automatic-check configurations at month -1: continuous integration (automated build and testing workflows), linters or formatters (checks for code problems or consistent formatting), commit hooks (programs triggered by commit actions), and test runners or coverage tools (tools that run tests or measure the code they exercise). The count ranges from zero to four. These variables record observable files and actions; test-file share measures files rather than test coverage, and a configuration file records a configured check.

We standardize each component using its mean and standard deviation across available project–cohort observations, then average the available standardized values when at least three components are observed. We split at the detected projects' median index, assigning equality to the weaker half and using the same cutoff for comparison observations. There are 238 detected projects in the weaker half and 237 in the stronger half. Within each half, we solve the primary entropy-balancing problem again and estimate the same event study as in the main paper. Density responses use that subgroup's own detected-project mean over months -6 to -1. Table 14 reports the initial practices in each half.

Tables 15 and 16 report the responses in percentages and native units, together with reference means, standard errors, and sample support. The stronger-minus-weaker contrasts subtract the separately normalized responses. Their reported standard error is $\sqrt{s_H^2 + s_L^2 - 2c_{HL}}$, where s_H and s_L are the standard errors from the subgroup fits and c_{HL} is their covariance, computed by aligning the repository-cluster score contributions across fits. The test uses a normal reference distribution. Figure 15 repeats the event study using later-detected comparisons only.

The weaker half's total-issue and technical-debt pre-period slope tests have $p = 0.81$ and

$p = 0.72$. The weaker half's bug series rises before detection; its slope test has $p = 0.04$, while its joint lead test has $p = 0.23$. All of these refer to the same monthly paths displayed in Figure 10.

Table 14: Development Practices before First Detected AI Use

	Weaker discipline	Stronger discipline
Human proposals reviewed (%)	36.190	74.270
Commits integrated through proposals (%)	45.660	85.290
Test files among code files (%)	16.500	31.880
Code changes also changing tests (%)	22.830	47.560
Configured automatic-check types (0–4)	2.210	2.930

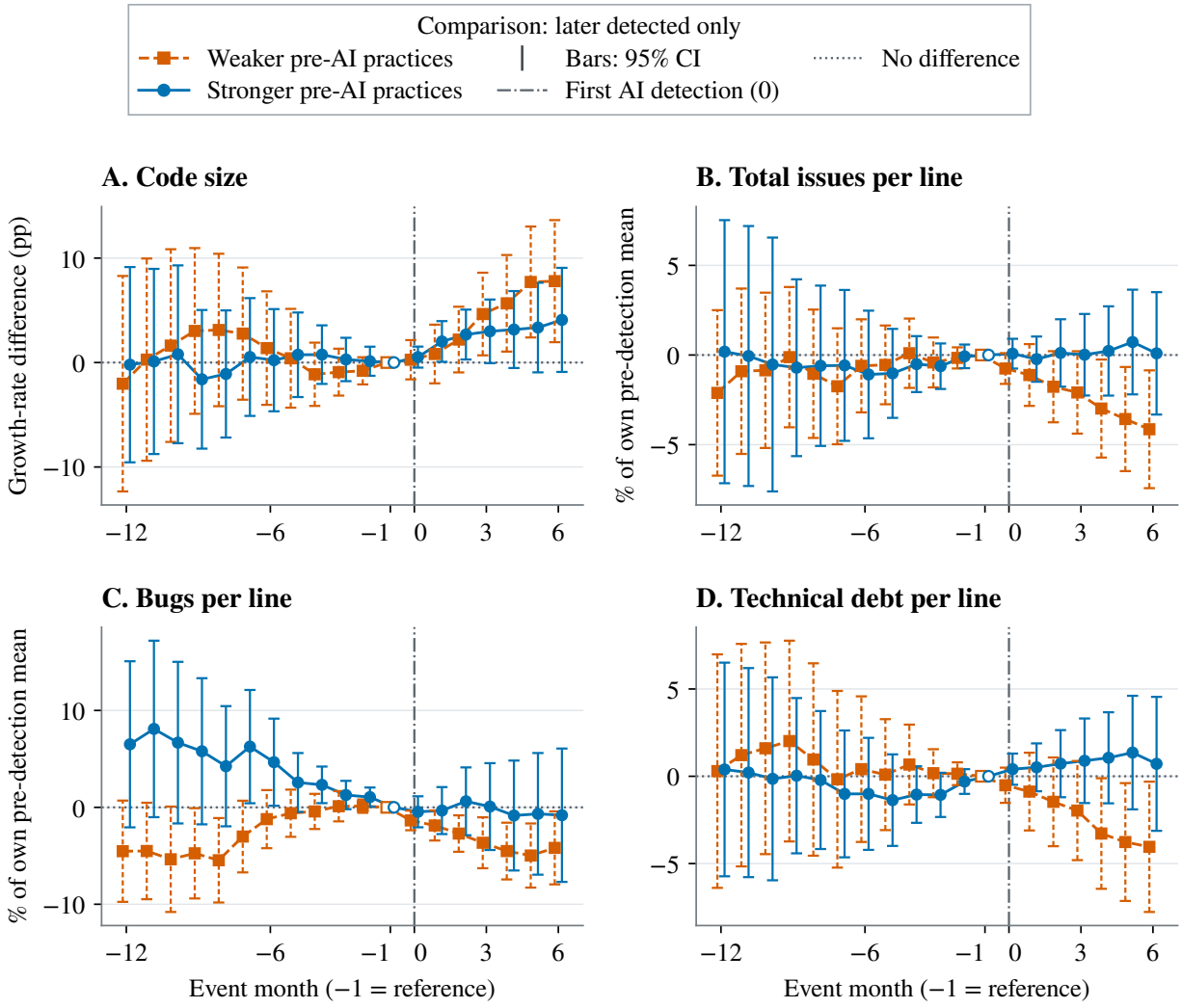
Detected-project means within the five-practice index halves. Stocks use month −1; flow shares use months −12 to −1. Baseline human reviews must occur within thirty days of opening and before month 0. Each component's mean uses the detected projects for which that component is observed; the index requires at least three components.

Table 15: Engineering Quality by Pre-AI Discipline (Percent of Own Pre-Detection Mean)

	Total issues	Bugs	Technical debt
Panel A: Later- or never-detected comparisons			
Weaker discipline	-2.095%	-3.047%	-1.846%
	(0.671)	(0.934)	(0.629)
p-value	< 0.01	< 0.01	< 0.01
Detected projects	238	238	238
Repository clusters	782	782	782
Stronger discipline	-0.170%	-0.111%	0.704%
	(0.533)	(1.124)	(0.609)
p-value	0.75	0.92	0.24
Detected projects	237	237	237
Repository clusters	501	501	501
Stronger minus weaker	1.926%	2.936%	2.550%
	(0.856)	(1.461)	(0.875)
p-value (normal test)	0.02	0.04	< 0.01
Panel B: Later-detected only comparisons			
Weaker discipline	-2.346%	-3.322%	-2.272%
	(1.096)	(1.163)	(1.330)
p-value	0.03	< 0.01	0.08
Detected projects	145	145	145
Repository clusters	335	335	335
Stronger discipline	0.146%	-0.346%	0.808%
	(1.048)	(2.120)	(1.117)
p-value	0.88	0.87	0.46
Detected projects	159	159	159
Repository clusters	303	303	303
Stronger minus weaker	2.492%	2.976%	3.081%
	(1.516)	(2.419)	(1.736)
p-value (normal test)	0.10	0.21	0.07

Responses are percentages of each subgroup's own detected-project mean over months −6 to −1, averaged over months 0 to +6 relative to month −1. Weights on size, total-issue and technical-debt pre-paths are recomputed within each half. Standard errors in parentheses are clustered by repository; counts are printed above. The difference test subtracts the separately normalized responses and uses $\sqrt{s_H^2 + s_L^2 - 2c_{HL}}$ with a normal reference distribution, where c_{HL} is the covariance obtained by aligning the repository-cluster score contributions from the two fits.

Figure 15: Pre-AI Discipline with Later-Detected Comparisons



Notes: The construction is as in Figure 10, restricting comparisons to later-detected projects in the same half. Bars are 95% intervals from repository-clustered standard errors.

Table 16: Engineering Quality by Pre-AI Discipline in Native Units

	Issues / 1,000 lines	Bugs / 1,000 lines	Technical debt ($/line)
Panel A: Later- or never-detected comparisons			
Weaker discipline: change	-0.558	-0.046	-0.118
	(0.179)	(0.014)	(0.040)
Pre-detection mean	26.637	1.521	6.408
Stronger discipline: change	-0.029	-0.001	0.031
	(0.090)	(0.009)	(0.027)
Pre-detection mean	16.820	0.761	4.472
Panel B: Later-detected only comparisons			
Weaker discipline: change	-0.671	-0.051	-0.156
	(0.313)	(0.018)	(0.091)
Pre-detection mean	28.593	1.540	6.851
Stronger discipline: change	0.023	-0.003	0.034
	(0.163)	(0.017)	(0.047)
Pre-detection mean	15.585	0.786	4.167

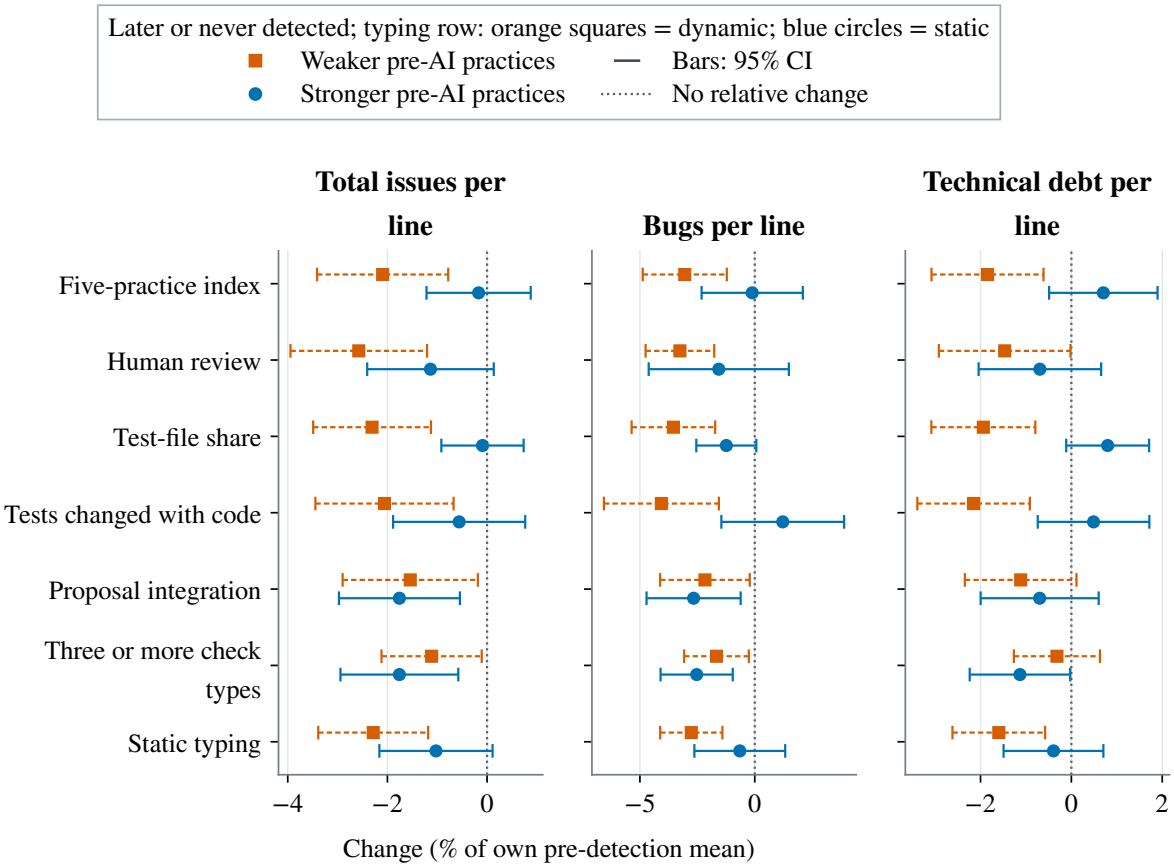
These are the contrasts in Table 15 multiplied by their own pre-detection means and divided by 100; standard errors are rescaled identically. Technical debt uses the paper's fixed dollar calibration.

D.2 Individual Practices, Balancing Variables, and Codebase Size

We first examine which practices distinguish the quality responses by splitting each component separately (Figure 16 and Table 17). The comparison for configured checks distinguishes at least three of the four types from fewer than three. The language comparison distinguishes statically typed languages, which check data types before execution, from dynamically typed languages, which check them during execution. Typing is not a component of the five-practice index. Each split retains its own observed sample and recomputes comparison weights within the group.

We then repeat the index comparison under alternative balancing variables and within smaller and larger codebases (Figure 17 and Table 18). The size cutoff is the detected projects' median average code size over months -12 to -1. The testing splits show the clearest quality-response differences, and the weaker-practice issue decline appears on both sides of the codebase-size cutoff.

Figure 16: Quality Responses across Individual Pre-AI Practices and Language Types



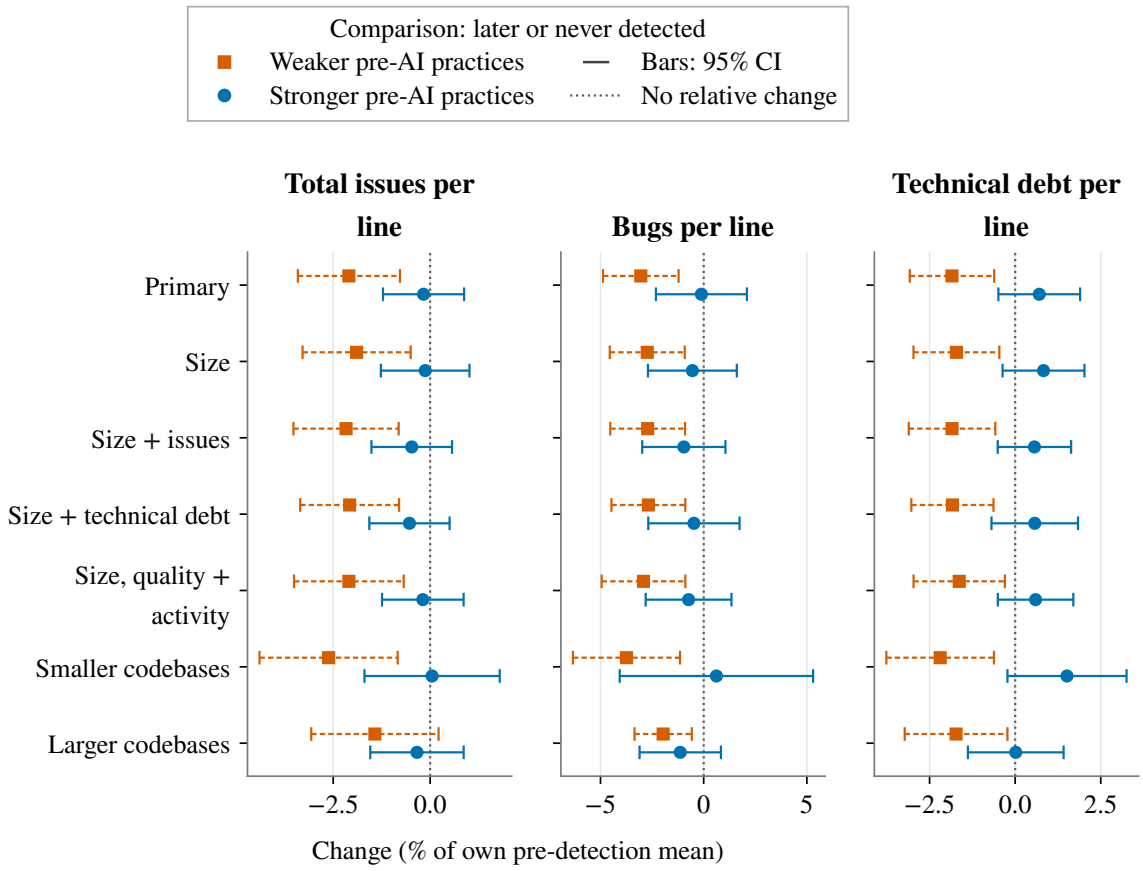
Notes: Orange points denote the weaker-practice or dynamically typed group, and blue points denote the stronger-practice or statically typed group, with later- or never-detected comparisons. Effects average months 0 to +6 relative to month −1, in percentages of each group's own pre-detection mean, with 95% intervals from repository-clustered standard errors.

Table 17: Quality-Response Differences across Individual Practices and Language Types

	Total issues	Bugs	Technical debt
Human review: stronger minus weaker			
Response difference	1.439%	1.692%	0.775%
	(0.954)	(1.732)	(1.010)
p-value	0.13	0.32	0.44
Test-file share: stronger minus weaker			
Response difference	2.216%	2.305%	2.736%
	(0.732)	(1.134)	(0.742)
p-value	< 0.01	0.04	< 0.01
Tests with code: stronger minus weaker			
Response difference	1.498%	5.284%	2.641%
	(0.980)	(1.864)	(0.887)
p-value	0.12	< 0.01	< 0.01
Proposal integration: stronger minus weaker			
Response difference	-0.217%	-0.494%	0.418%
	(0.927)	(1.444)	(0.910)
p-value	0.81	0.73	0.64
Automatic-check types: stronger minus weaker			
Response difference	-0.647%	-0.863%	-0.814%
	(0.794)	(1.076)	(0.744)
p-value	0.41	0.42	0.27
Statically typed minus dynamically typed languages			
Response difference	1.259%	2.111%	1.204%
	(0.808)	(1.223)	(0.765)
p-value	0.11	0.08	0.11

Later- or never-detected comparisons. Each practice is split separately using its own observed sample and detected-project median; the automatic-check indicator compares at least three check types with fewer than three, and the typing comparison subtracts dynamically typed projects from statically typed projects. Responses and normal difference tests follow Table 15; positive differences indicate larger declines in the weaker-practice group, or in dynamically typed projects for the typing comparison.

Figure 17: Discipline Heterogeneity across Balancing Variables and Codebase Sizes



Notes: Orange points denote weaker discipline and blue points stronger discipline; comparisons are later- or never-detected projects in the same group. Effects and intervals follow Figure 16; weights are recomputed for each row.

Table 18: Pre-AI Discipline under Alternative Balancing Variables and Codebase Sizes

	Total issues	Bugs	Technical debt
Size only			
Weaker discipline	-1.896%	-2.734%	-1.716%
	(0.711)	(0.927)	(0.640)
Stronger discipline	-0.128%	-0.548%	0.828%
	(0.583)	(1.099)	(0.610)
Size and total issues			
Weaker discipline	-2.168%	-2.714%	-1.844%
	(0.692)	(0.924)	(0.645)
Stronger discipline	-0.474%	-0.962%	0.563%
	(0.530)	(1.029)	(0.547)
Size and technical debt			
Weaker discipline	-2.075%	-2.681%	-1.833%
	(0.651)	(0.913)	(0.612)
Stronger discipline	-0.533%	-0.473%	0.572%
	(0.528)	(1.128)	(0.645)
Size, quality and activity			
Weaker discipline	-2.096%	-2.917%	-1.635%
	(0.721)	(1.034)	(0.680)
Stronger discipline	-0.188%	-0.732%	0.596%
	(0.537)	(1.061)	(0.563)
Smaller codebases			
Weaker discipline	-2.618%	-3.741%	-2.191%
	(0.909)	(1.324)	(0.802)
Stronger discipline	0.051%	0.617%	1.514%
	(0.890)	(2.391)	(0.887)
Larger codebases			
Weaker discipline	-1.424%	-1.964%	-1.726%
	(0.837)	(0.707)	(0.765)
Stronger discipline	-0.339%	-1.136%	0.017%
	(0.614)	(1.006)	(0.713)

Later- or never-detected comparisons; responses are percentages of each group's own pre-detection mean. The first four panels change the balancing variables; the final two split codebases at the detected projects' median pre-year size, then recompute weights within each size and discipline group. Standard errors are clustered by repository.

D.3 Testing and Human Development Procedures

The testing and procedure event studies use changes since month −1 in native units, without winsorization. Table 19 reports testing, proposal integration, and configured checks in both comparison groups. Figure 18 shows the monthly paths for configured checks, integration through proposals, human review, and the presence of files defining contribution procedures. The latter counts contribution instructions, proposal templates, and files assigning review responsibility.

For the commit comparison, we retain non-bot code-changing first-parent commits in months 0 to +6. We distinguish commits carrying a recorded AI credit or coding-agent author from unmarked commits. The adjusted comparison uses project–month fixed effects, bins of commit size, and project–months containing both kinds of commits. It therefore uses a smaller sample than the raw test-changing rates. In the weaker half, the adjusted comparison uses 10,051 commits in 81 projects. AI-marked contributions account for 3.9% of all observed code-changing commits in that half. Standard errors are clustered by project.

The within-project regression of issue density on test-file share includes project and calendar-month fixed effects and the natural log of code size. Its percentage coefficient uses the mean of this project–month panel, 30.202 total issues per thousand lines. The coefficient-adjustment exercise then adds the change in test-file share since month −1 to each subgroup's quality event study. Table 20 holds the observations fixed between the unadjusted and adjusted fits. Test files are included in the scanned codebase, so the change in its composition is included in these quality measures.

Table 21 separates the broad review/sign-off/coauthor-credit marker into AI or bot credits and reviewer acknowledgments or sign-offs. It also reports references to tracked issues and proposals, integration practices, work timing, and review of human-authored proposals. These shares measure the prevalence of actions or credits, whereas the main paper's reviewed-proposal outcome counts reviewed proposals. The distinction allows proposal volume to increase without an increase in its reviewed fraction.

Table 19: Changes in Testing and Development Practices by Pre-AI Discipline

	Weaker	Stronger
Later- or never-detected comparisons		
Test-file share	0.800	0.038
	(0.307)	(0.250)
p-value	< 0.01	0.88
Detected / repository clusters	238 / 782	237 / 501
Tests changed with code	2.955	4.865
	(2.039)	(2.058)
p-value	0.14	0.01
Detected / repository clusters	175 / 624	203 / 459
Automatic-check types	0.064	-0.015
	(0.020)	(0.014)
p-value	< 0.01	0.26
Detected / repository clusters	238 / 782	237 / 501
Proposal integration	0.987	-0.822
	(1.866)	(1.336)
p-value	0.59	0.53
Detected / repository clusters	186 / 658	216 / 466
Later-detected only comparisons		
Test-file share	0.787	-0.746
	(0.381)	(0.470)
p-value	0.03	0.11
Detected / repository clusters	145 / 335	159 / 303
Tests changed with code	0.903	1.141
	(2.972)	(2.238)
p-value	0.76	0.61
Detected / repository clusters	112 / 290	145 / 289
Automatic-check types	0.044	-0.008
	(0.039)	(0.026)
p-value	0.26	0.77
Detected / repository clusters	145 / 335	159 / 303
Proposal integration	2.153	-1.022
	(3.269)	(1.736)
p-value	0.51	0.55
Detected / repository clusters	120 / 302	150 / 291

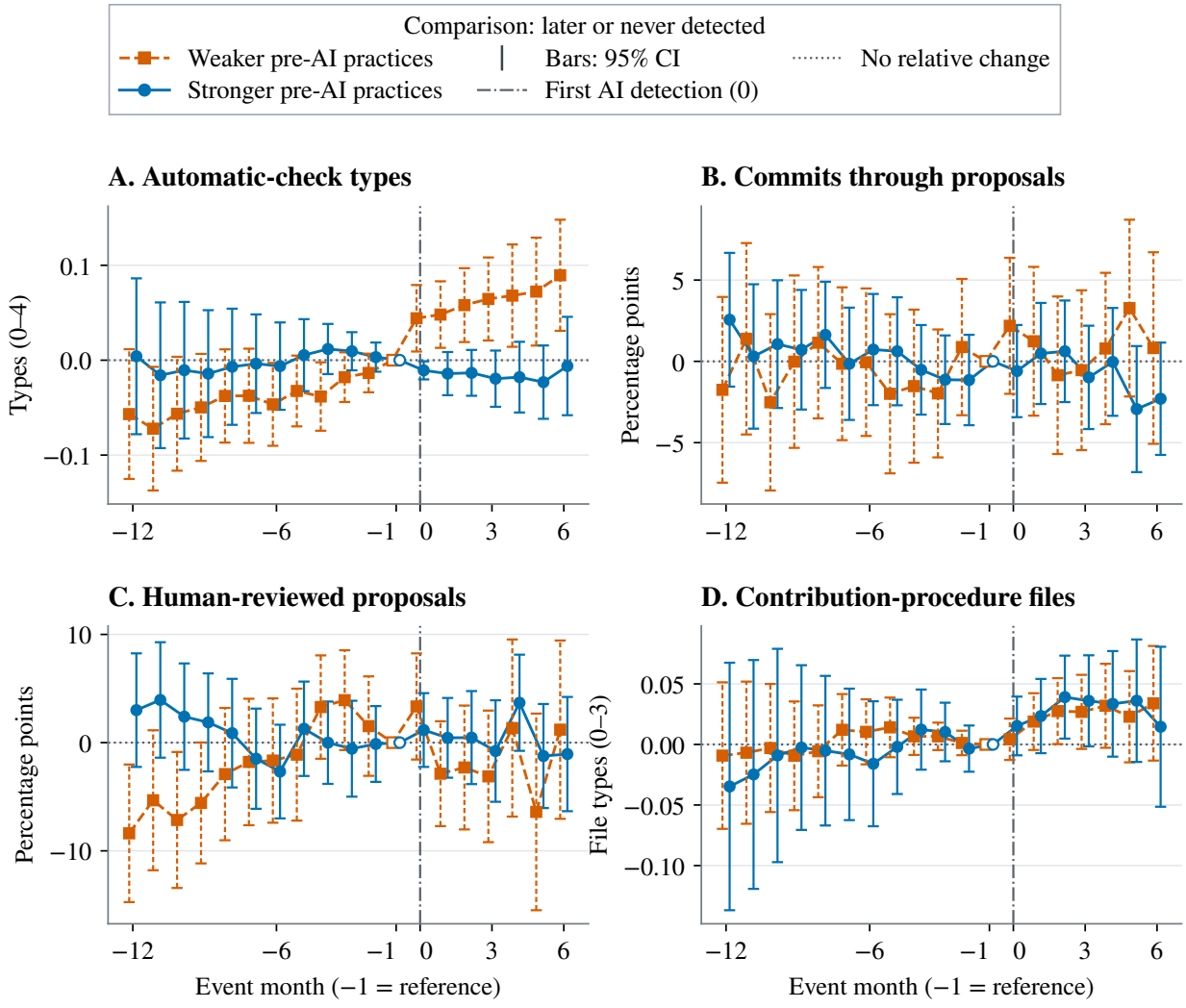
Shares are measured in percentage points and automatic checks in configured types (0–4). Outcomes are unwinsorized changes since month -1; weights are recomputed within each half. Standard errors are clustered by repository. The test-file share counts files, and tests changed with code counts code-changing first-parent commits that also change test files.

Table 20: Quality Responses with the Change in Test-File Share Held Fixed

	Total issues	Bugs	Technical debt
Weaker discipline			
Without test-share adjustment	-2.083%	-3.034%	-1.838%
	(0.668)	(0.932)	(0.627)
With test-share adjustment	-1.683%	-2.853%	-1.423%
	(0.674)	(0.931)	(0.645)
Reduction in coefficient magnitude	19.225%	5.967%	22.583%
Stronger discipline			
Without test-share adjustment	-0.172%	-0.110%	0.704%
	(0.533)	(1.124)	(0.609)
With test-share adjustment	-0.151%	-0.082%	0.722%
	(0.515)	(1.156)	(0.594)

Later- or never-detected comparisons with the same observations in each unadjusted/adjusted pair; each percentage response uses the original subgroup's pre-detection mean. The additional regressor is the change in test-file share since month -1. Standard errors are clustered by repository.

Figure 18: Human Review and Contribution Procedures by Pre-AI Discipline



Notes: Later- or never-detected comparisons use weights recomputed within each half and month −1 as the reference. Points show unwinsorized monthly responses with 95% intervals from repository-clustered standard errors.

Table 21: Human Review and Contribution Procedures

	Change (pp)	SE	p	Detected	Clusters
Panel A: Commit shares					
Any review/sign-off/coauthor credit	4.091	(0.960)	< 0.01	425	1204
AI or bot coauthor credit	5.320	(0.586)	< 0.01	425	1204
Reviewer/sign-off/acknowledgment	-0.460	(0.540)	0.39	425	1204
All credits, unmarked commits	-0.647	(0.831)	0.43	424	1204
Issue reference	-0.222	(1.516)	0.88	425	1204
Proposal reference	-0.601	(1.425)	0.67	425	1204
Merge/squash integration	0.048	(1.439)	0.97	425	1204
Outside 06:00–20:00 UTC	0.538	(1.155)	0.64	425	1204
Weekend	0.173	(0.873)	0.84	425	1204
Panel B: Human proposal shares					
Human-reviewed proposals	-1.611	(1.419)	0.25	376	751
Merged proposals with zero reviews	1.116	(1.358)	0.41	362	700
Changes requested	-0.130	(0.669)	0.84	376	751
Merged within 30 days	-0.216	(1.482)	0.88	378	761
Proposals changing over 500 lines	3.279	(1.370)	0.01	378	761

Later- or never-detected comparisons. Shares are unwinsorized changes since month -1; standard errors are clustered by repository. Reviewer/sign-off/acknowledgment excludes coauthor-credit lines; unmarked commits have no recorded AI signature. Human-reviewed proposals and requests for changes use the thirty-day follow-up described in the data section. The zero-review measure is the share of merged, review-measurable human proposals with no recorded reviews at collection.

D.4 Excluding Recorded Investor Financing

We link GitHub owners to PitchBook company-financing records and apply each restriction to detected and comparison projects alike. The first restriction excludes equity rounds within the project's cohort window, months -12 to $+6$. The second also excludes acquisitions, buyouts, and initial public offerings in that window. The third excludes any equity round since 2023. The fourth excludes companies with any recorded equity round, retaining projects without a company link. The fifth applies the same exclusion while retaining only projects linked to companies in the financing data. Every restriction is followed by a new solution of the entropy-balancing weights.

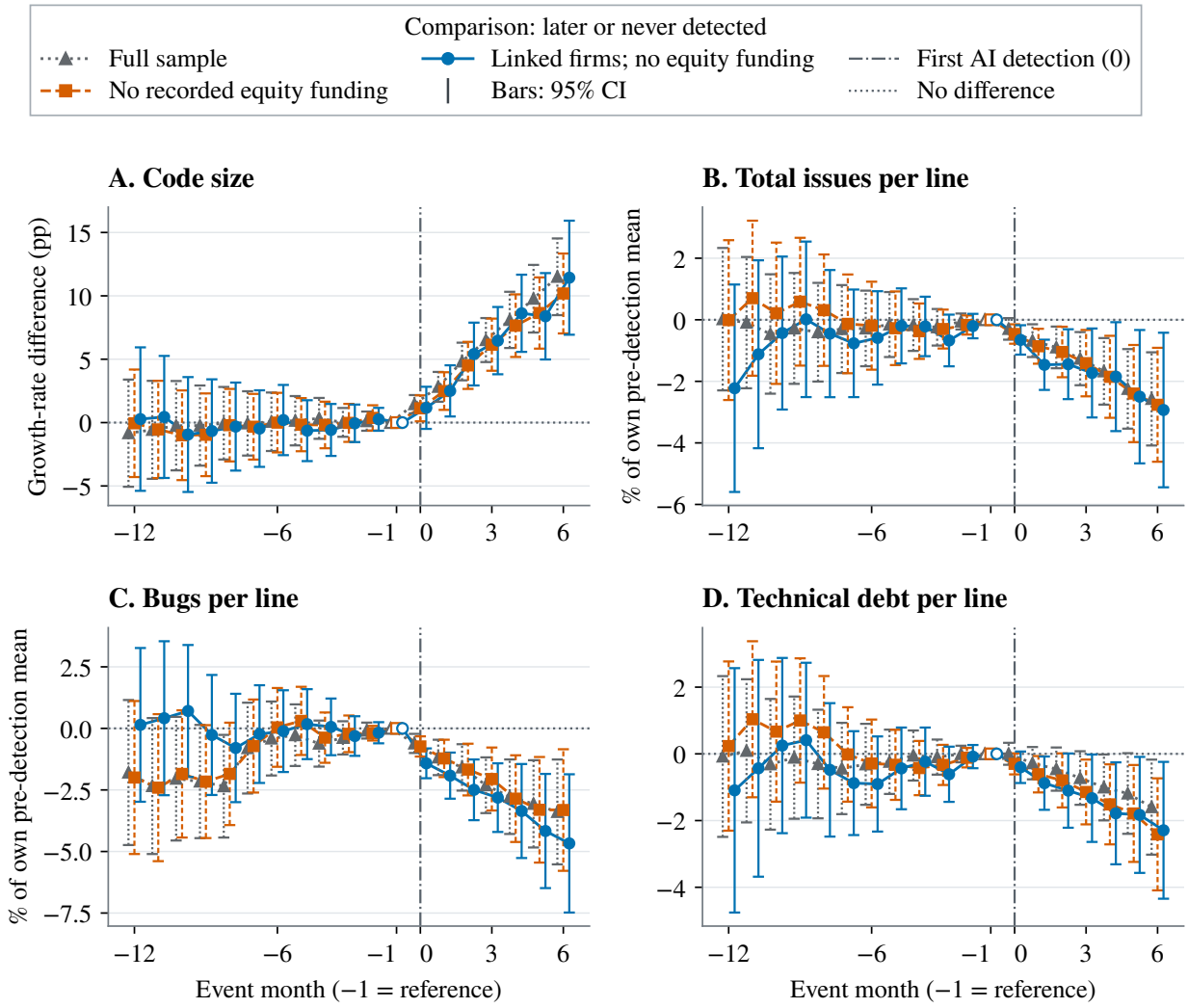
The transaction records cover deals through March 20, 2025, supplemented by company profiles recording each company's last funding round through October 2025. The broader company exclusions therefore complement the event-window exclusions. The linked-company restriction separately checks the treatment of projects without a financing-data link. Table 22 reports each financing restriction for both comparison groups. Figure 19 compares the monthly paths in the full sample with the two samples excluding all recorded equity funding. Total issues, bugs, and technical debt per line continue to decline after these exclusions.

Table 22: Engineering Quality and Code Growth after Funding Exclusions

	Code size (pp)	Total issues	Bugs	Technical debt	Detected
Full sample					
Later- or never-detected	6.4	-1.375%	-2.131%	-0.750%	499
	(0.8)	(0.412)	(0.563)	(0.379)	
Later-detected only	3.3	-1.538%	-2.348%	-0.688%	322
	(1.2)	(0.633)	(0.898)	(0.676)	
No equity round in event window					
Later- or never-detected	6.0	-1.655%	-1.950%	-1.002%	464
	(0.9)	(0.402)	(0.567)	(0.388)	
Later-detected only	3.6	-1.786%	-2.303%	-0.974%	293
	(1.2)	(0.652)	(0.883)	(0.712)	
No equity round or ownership change					
Later- or never-detected	5.9	-1.618%	-1.897%	-0.992%	463
	(0.8)	(0.403)	(0.566)	(0.389)	
Later-detected only	3.6	-1.791%	-2.275%	-0.979%	292
	(1.2)	(0.658)	(0.891)	(0.717)	
No equity round since 2023					
Later- or never-detected	5.6	-1.661%	-1.940%	-1.147%	411
	(0.8)	(0.419)	(0.566)	(0.397)	
Later-detected only	3.3	-1.546%	-2.245%	-0.952%	258
	(1.1)	(0.719)	(0.915)	(0.777)	
No recorded equity funding					
Later- or never-detected	5.8	-1.541%	-2.166%	-1.223%	296
	(0.9)	(0.492)	(0.656)	(0.452)	
Later-detected only	3.4	-1.998%	-2.187%	-1.656%	184
	(1.3)	(0.810)	(1.256)	(0.737)	
Linked firms without equity funding					
Later- or never-detected	6.3	-1.795%	-2.978%	-1.374%	163
	(1.2)	(0.688)	(0.740)	(0.586)	
Later-detected only	5.6	-3.300%	-4.051%	-1.974%	94
	(1.7)	(1.374)	(1.267)	(1.294)	

Weights are recomputed after applying each funding restriction to detected and comparison projects alike. Code-size responses and their standard errors are 100 times natural-log differences, in percentage points. Quality responses are percentages of each subsample's own pre-detection mean, with repository-clustered standard errors. Equity includes seed, angel, accelerator, venture, growth, corporate and crowdfunding rounds; ownership changes include acquisitions, buyouts and IPOs.

Figure 19: Code Growth and Engineering Quality after Excluding Equity-Funded Firms



Notes: The full sample and both funding-restricted samples use later- or never-detected comparisons, with weights recomputed within each sample. Monthly responses are relative to month −1, quality uses each sample's own pre-detection mean, code size uses approximate percentage-point differences in growth, and bars show 95% intervals from repository-clustered standard errors.

E Engineering Burden Relative to Recorded Functionality

This appendix develops the measurement comparison in Section 4.4. It explains the cumulative feature count, compares matching and sample choices, and places the log specification alongside alternative measures of burden per unit of software functionality.

E.1 Recorded Feature Additions and the Log Outcome

We identify feature additions from the first line of each commit message. A declaration begins with `feat:`, optionally with a scope or a breaking-change marker, as in `feat(api): add search` or `feat!: revise export`. Matching is case-insensitive. We deduplicate commits by repository and commit identifier and exclude merge commits and commits by Dependabot, a tool that automatically proposes updates to software packages used by a project. We accumulate qualifying declarations through each month. An observed month with no feature declaration adds zero to the counter; a month without current message coverage is excluded from the ratio outcome. The resulting count records declared additions to functionality, without assigning a dollar value to them.

For burden B_{it}, either total issues or technical debt in dollars, the outcome is

$$y_{it} = \ln(B_{it}/F_{it}) = \ln B_{it} - \ln F_{it}. \tag{5}$$

We retain observed months with positive burdens, feature counts, and lines of code. These outcomes use ordinary logarithms, without an added constant, outcome standardization, or division by a pre-period mean. Outcomes are not winsorized or interpolated. The fixed conversion of repair time into dollars adds a constant to the technical-debt log outcome, which the fixed effects absorb.

We retain the project-by-cohort and cohort-by-calendar-month fixed effects of the main event-study specification. First detection combines the recorded AI signals. Later-detected comparisons remain undetected through month +6 of the relevant cohort window. If β_k denotes the coefficient for event month k, with $\beta_{-1} = 0$, the headline contrast is $\theta_{-1} = 7^{-1} \sum_{k=0}^{6} \beta_k$. The alternative reference uses $\theta_{\text{pre6}} = \theta_{-1} - 6^{-1} \sum_{k=-6}^{-1} \beta_k$. Each contrast uses the full fitted covariance matrix. Repository-clustered CRV1 inference uses a t distribution with $G - 1$ degrees of freedom and holds the matching weights fixed. We report 100θ as the approximate percentage-point difference in growth rates relative to comparison projects, and multiply both confidence endpoints by 100.

E.2 Matching Recorded Functional Development before Detection

The initial specification balances means and slopes over months −12 through −4 for five series: the log burden per feature, log feature count, log code size, total issues per thousand lines, and technical debt in dollars per line. It requires six observed training months for each series and an observed positive

ratio in month −1. We also balance cohort and language shares. This yields 188 detected projects in the pooled comparison.

The paper-path comparison balances the earlier paths of log code size and the two quality densities separately within each detection month. It uses the main paper's sequence of monthly, coarser, and minimal moment sets, with its ridge approximation when needed. We report this comparison both on the original eligible sample and after removing the feature-specific training-history requirement. The broader comparison retains 225 detected projects. Table 23 reports both outcomes and all three comparison groups for these choices.

Table 23: Engineering Burden per Recorded Feature

Outcome	Log change	SE	Growth gap (pp)	p	Detected	Clusters
Later or never detected						
Feature-history matching						
Total issues	-0.067	0.031	-6.70	0.03	188	403
Technical debt	-0.050	0.031	-5.04	0.10	188	403
Paper paths, original sample						
Total issues	-0.074	0.033	-7.40	0.02	188	398
Technical debt	-0.055	0.034	-5.51	0.10	188	398
Paper paths, broader sample						
Total issues	-0.060	0.036	-5.99	0.09	225	525
Technical debt	-0.044	0.037	-4.44	0.22	225	525
Later detected only						
Feature-history matching						
Total issues	-0.070	0.032	-7.00	0.02	126	248
Technical debt	-0.055	0.032	-5.54	0.08	126	248
Paper paths, original sample						
Total issues	-0.081	0.030	-8.11	< 0.01	126	245
Technical debt	-0.069	0.030	-6.90	0.02	126	245
Paper paths, broader sample						
Total issues	-0.062	0.032	-6.16	0.05	147	292
Technical debt	-0.044	0.032	-4.38	0.17	147	292
Never detected						
Feature-history matching						
Total issues	-0.068	0.032	-6.79	0.03	188	323
Technical debt	-0.045	0.034	-4.50	0.18	188	323
Paper paths, original sample						
Total issues	-0.076	0.039	-7.56	0.05	188	321

Table 23 continued

Outcome	Log change	SE	Growth gap (pp)	p	Detected	Clusters
Technical debt	-0.050	0.041	-4.99	0.22	188	321
Paper paths, broader sample						
Total issues	-0.094	0.033	-9.40	< 0.01	225	430
Technical debt	-0.072	0.034	-7.16	0.03	225	430

Notes: The outcome is the natural log of total issues or technical debt in dollars divided by cumulative feature-labeled commits. The log change averages months 0 through 6 relative to month -1; the approximate growth-rate difference in percentage points is 100θ. Standard errors are repository-clustered CRV1, with a t_{G-1} reference and matching weights held fixed. The feature-history and both paper-path samples are defined in the text. Reported p-values test the individual contrast.

Table 24: Matching Support and Achieved Balance

Outcome	Detected	Comparisons	ESS	Pooled gap	Cohort gap	Approx.
Later or never detected						
Feature-history matching						
Total issues	188	319	79.6	0.000	—	0
Technical debt	188	319	80.5	0.000	—	0
Paper paths, original sample						
Total issues	188	314	71.0	0.019	0.348	39
Technical debt	188	314	71.4	0.019	0.348	39
Paper paths, broader sample						
Total issues	225	420	96.1	0.005	0.352	8
Technical debt	225	420	95.9	0.005	0.352	8
Later detected only						
Feature-history matching						
Total issues	126	184	43.5	0.000	—	0
Technical debt	126	184	42.4	0.000	—	0
Paper paths, original sample						
Total issues	126	181	40.1	0.059	0.346	54
Technical debt	126	181	40.1	0.059	0.346	54
Paper paths, broader sample						
Total issues	147	216	45.5	0.060	0.350	51
Technical debt	147	216	45.5	0.060	0.350	51
Never detected						
Feature-history matching						
Total issues	188	135	32.9	0.000	—	0
Technical debt	188	135	33.0	0.000	—	0
Paper paths, original sample						
Total issues	188	133	37.3	0.106	0.901	109
Technical debt	188	133	37.4	0.106	0.901	109
Paper paths, broader sample						
Total issues	225	205	44.9	0.072	0.902	63
Technical debt	225	205	44.8	0.072	0.902	63

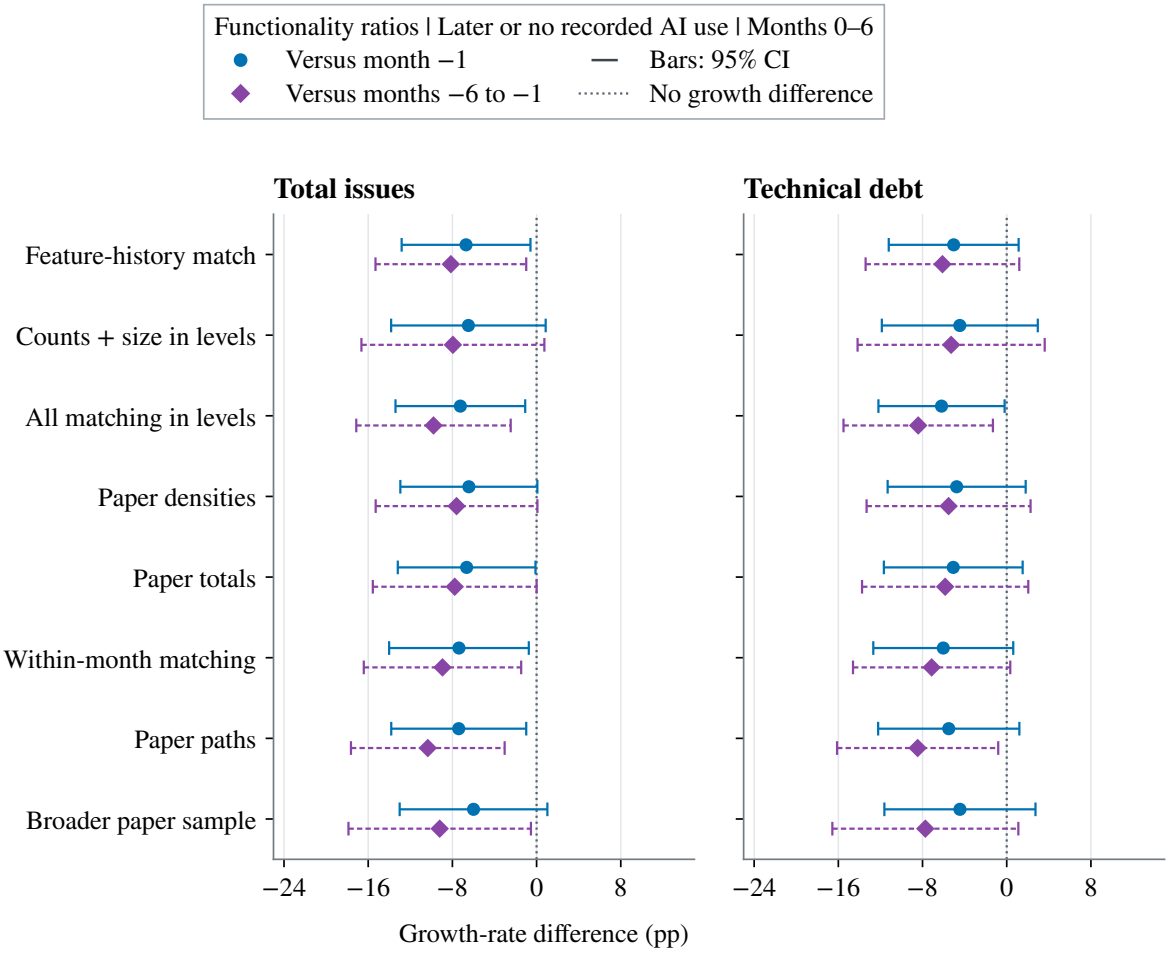
Comparison counts are unique repositories. ESS is $(\sum_j w_j)^2/\sum_j w_j^2$ after aggregating comparison weights by repository. Gaps are maximum absolute standardized differences on the fitted matching moments; the cohort column gives the largest cohort-specific gap where matching is separate by month. Approx. counts detected projects in cohorts using the paper's ridge approximation. The original pooled matcher uses the fixed 0.05 gap criterion; paper-path matching follows its original sequence and fallback.

Figure 20 compares feature-history matching with alternatives that use feature counts and size in levels, all matching variables in levels, the paper's quality densities or burden totals, and matching separately within detection month. The final two comparisons use main-analysis path matching on the

feature-history and broader samples. The cohort-specific procedures retain the paper's approximate fallback; feature-history matching instead requires a maximum absolute standardized gap below 0.05. Table 24 reports comparison support, the effective number of weighted comparison repositories, and achieved balance. Pooled and within-cohort gaps are reported separately because a close aggregate match can coexist with larger gaps in individual detection cohorts.

All 46 available nonredundant month-minus-one contrasts across the matching specifications and comparison groups are negative. Using the six-month pre-detection reference, 45 are negative. The exception uses technical debt, matching on burden totals, and never-detected comparisons; its confidence interval includes zero. In the broader pooled sample, the six-month reference gives growth differences of approximately -9.2 percentage points in total issues per unit of functionality ($p = 0.03$) and -7.7 percentage points in technical debt per unit of functionality ($p = 0.08$). Changing this reference changes the contrast, rather than only its reporting units. Table 25 reports these matching comparisons.

Figure 20: Feature Outcomes across Matching Specifications



Notes: Both panels report the average response over months 0–6. Blue circles use month −1 as the reference; purple diamonds use the average of months −6 to −1. The outcome is the log burden per cumulative recorded feature, displayed as 100θ, an approximate growth-rate difference in percentage points; interval endpoints use the same scaling. Intervals use the full covariance of the relevant contrast.

Table 25: Feature Outcomes across Matching Specifications

Choice	Issues: pp [95% CI]	p	N	TD: pp [95% CI]	p	N
Later or never detected						
Feature-history matching	-6.70 [-12.82, -0.58]	0.03	188	-5.04 [-11.21, 1.13]	0.10	188
Feature count and size in levels	-6.48 [-13.82, 0.86]	0.08	188	-4.46 [-11.87, 2.95]	0.23	188
All matching variables in levels	-7.24 [-13.41, -1.08]	0.02	188	-6.19 [-12.19, -0.20]	0.04	188
Paper variables: densities	-6.44 [-12.95, 0.07]	0.05	188	-4.76 [-11.31, 1.80]	0.15	188
Paper variables: totals	-6.65 [-13.19, -0.10]	0.04	188	-5.08 [-11.67, 1.51]	0.13	188
Match within detection month	-7.38 [-14.01, -0.74]	0.02	188	-6.03 [-12.68, 0.62]	0.07	188
Paper paths, original sample	-7.40 [-13.81, -0.98]	0.02	188	-5.51 [-12.22, 1.20]	0.10	188
Paper paths, broader sample	-5.99 [-13.01, 1.02]	0.09	225	-4.44 [-11.62, 2.73]	0.22	225
Later detected only						
Feature-history matching	-7.00 [-13.22, -0.78]	0.02	126	-5.54 [-11.88, 0.79]	0.08	126
Feature count and size in levels	-6.96 [-14.40, 0.47]	0.06	126	-6.71 [-14.48, 1.07]	0.09	126
All matching variables in levels	-6.09 [-12.75, 0.57]	0.07	126	-5.86 [-12.45, 0.73]	0.08	126
Paper variables: densities	-7.49 [-13.74, -1.24]	0.01	126	-6.41 [-12.78, -0.04]	0.04	126
Paper variables: totals	-7.01 [-13.06, -0.95]	0.02	126	-5.59 [-11.65, 0.47]	0.07	126
Match within detection month	-7.28 [-13.14, -1.43]	0.01	126	-6.48 [-12.58, -0.38]	0.03	126
Paper paths, original sample	-8.11 [-13.93, -2.29]	< 0.01	126	-6.90 [-12.88, -0.92]	0.02	126
Paper paths, broader sample	-6.16 [-12.41, 0.08]	0.05	147	-4.38 [-10.72, 1.95]	0.17	147
Never detected						
Feature-history matching	-6.79 [-13.16, -0.42]	0.03	188	-4.50 [-11.16, 2.17]	0.18	188

Table 25 continued

Choice	Issues: pp [95% CI]	p	N	TD: pp [95% CI]	p	N
Feature count and size in levels	-8.74 [-18.25, 0.77]	0.07	188	-5.33 [-14.17, 3.52]	0.23	188
Paper variables: densities	-5.98 [-12.99, 1.04]	0.09	188	-3.70 [-11.00, 3.61]	0.32	188
Paper variables: totals	-4.84 [-11.35, 1.67]	0.14	188	-2.83 [-9.72, 4.05]	0.41	188
Match within detection month	-8.53 [-15.67, -1.39]	0.01	188	-6.28 [-13.44, 0.88]	0.08	188
Paper paths, original sample	-7.56 [-15.17, 0.06]	0.05	188	-4.99 [-13.02, 3.04]	0.22	188
Paper paths, broader sample	-9.40 [-15.91, -2.88]	< 0.01	225	-7.16 [-13.84, -0.48]	0.03	225

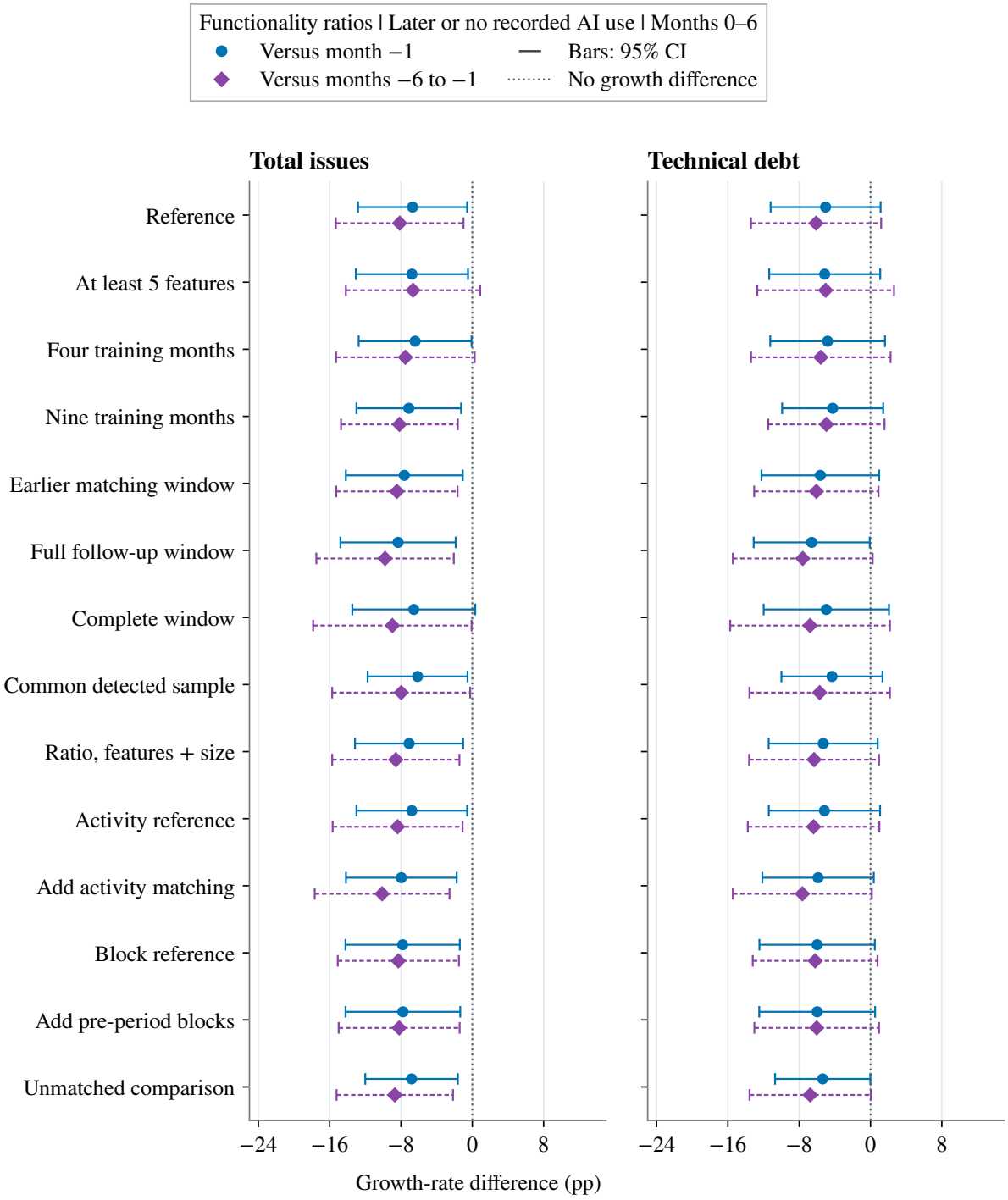
Notes: Each cell reports the approximate growth-rate difference 100θ in percentage points for the average of months 0–6 relative to month -1, followed by the equally scaled 95% confidence interval. N is the number of detected projects. TD denotes technical debt in dollars per cumulative recorded feature.

E.3 Sample Coverage and Matching Windows

The sample comparisons raise the minimum feature count, relax or tighten the required training history, and move the matching window earlier. Other comparisons retain cohorts with full potential follow-up, require a complete observed window, or hold the detected-project sample common across outcomes. We also add activity measures or pre-period block means to the matching, comparing each with a reference restricted to the same covered sample. The complete-window restriction is a support diagnostic; the unmatched comparison is reported separately from the matched estimates.

All 13 available matched pooled specifications for each outcome have negative month-minus-one point estimates. Across the separate comparison groups, technical debt reverses sign under the strict nine-month training requirement and never-detected comparisons; this estimate is not statistically significant ($p = 0.17$). Figure 21 displays the pooled comparisons under both reference periods. Table 26 reports the estimates for all three comparison groups.

Figure 21: Feature Outcomes across Sample Choices



Notes: Both panels report the average response over months 0–6. Blue circles use month −1 as the reference; purple diamonds use the average of months −6 to −1. The outcome is the log burden per cumulative recorded feature, displayed as 100θ, an approximate growth-rate difference in percentage points; interval endpoints use the same scaling. Intervals use the full covariance of the relevant contrast.

Table 26: Feature Outcomes across Sample Choices

Choice	Issues: pp [95% CI]	*p*	*N*	TD: pp [95% CI]	*p*	*N*
Later or never detected						
Reference	-6.70 [-12.82, -0.58]	0.03	188	-5.04 [-11.21, 1.13]	0.10	188
At least five feature commits	-6.77 [-13.07, -0.48]	0.03	154	-5.14 [-11.37, 1.09]	0.10	154
Four training months	-6.42 [-12.75, -0.09]	0.04	201	-4.81 [-11.25, 1.62]	0.14	201
Nine training months	-7.13 [-13.00, -1.26]	0.01	126	-4.25 [-9.92, 1.42]	0.14	126
Earlier matching window	-7.63 [-14.19, -1.07]	0.02	181	-5.63 [-12.23, 0.97]	0.09	181
Full follow-up window	-8.33 [-14.79, -1.86]	0.01	143	-6.60 [-13.11, -0.09]	0.04	143
Complete observed window	-6.57 [-13.46, 0.33]	0.06	122	-4.96 [-11.99, 2.07]	0.16	122
Common detected sample	-6.14 [-11.74, -0.54]	0.03	126	-4.32 [-9.99, 1.34]	0.13	126
Match ratio, features and size	-7.09 [-13.16, -1.03]	0.02	188	-5.31 [-11.42, 0.81]	0.08	188
Activity-covered reference	-6.79 [-12.99, -0.58]	0.03	183	-5.17 [-11.41, 1.07]	0.10	183
Add activity matching	-7.96 [-14.17, -1.75]	0.01	183	-5.88 [-12.13, 0.36]	0.06	183
Block-covered reference	-7.81 [-14.23, -1.40]	0.01	181	-5.99 [-12.46, 0.48]	0.06	181
Add pre-period blocks	-7.79 [-14.22, -1.35]	0.01	181	-5.98 [-12.49, 0.52]	0.07	181
Unmatched comparison	-6.81 [-12.01, -1.62]	0.01	188	-5.36 [-10.71, -0.02]	0.04	188
Later detected only						
Reference	-7.00 [-13.22, -0.78]	0.02	126	-5.54 [-11.88, 0.79]	0.08	126
Four training months	-6.08 [-12.45, 0.29]	0.06	134	-4.17 [-10.75, 2.41]	0.21	134
Earlier matching window	-7.48 [-13.52, -1.45]	0.01	120	-6.05 [-12.24, 0.14]	0.05	120
Full follow-up window	-7.00 [-13.22, -0.78]	0.02	126	-5.54 [-11.88, 0.79]	0.08	126
Complete observed window	-6.25 [-12.56, 0.06]	0.05	109	-4.01 [-10.82, 2.80]	0.24	109
Common detected sample	-7.00 [-13.22, -0.78]	0.02	126	-5.54 [-11.88, 0.79]	0.08	126
Match ratio, features and size	-7.40 [-13.50, -1.30]	0.01	126	-6.34 [-12.61, -0.06]	0.04	126

Table 26 continued

Choice	Issues: pp [95% CI]	p	N	TD: pp [95% CI]	p	N
Activity-covered reference	-7.12 [-13.38, -0.87]	0.02	125	-5.64 [-12.00, 0.73]	0.08	125
Add activity matching	-7.73 [-14.04, -1.42]	0.01	125	-6.57 [-13.08, -0.05]	0.04	125
Block-covered reference	-7.74 [-13.66, -1.82]	0.01	121	-6.19 [-12.28, -0.10]	0.04	121
Add pre-period blocks	-7.71 [-13.64, -1.77]	0.01	121	-6.13 [-12.20, -0.05]	0.04	121
Unmatched comparison	-5.88 [-11.59, -0.17]	0.04	126	-4.66 [-10.50, 1.18]	0.11	126
Never detected						
Reference	-6.79 [-13.16, -0.42]	0.03	188	-4.50 [-11.16, 2.17]	0.18	188
At least five feature commits	-7.79 [-14.27, -1.32]	0.01	154	-5.50 [-12.01, 1.00]	0.09	154
Four training months	-6.49 [-13.35, 0.37]	0.06	201	-4.67 [-11.78, 2.45]	0.19	201
Nine training months	Not estimated	—	—	7.36 [-3.22, 17.94]	0.17	126
Earlier matching window	-7.93 [-14.78, -1.07]	0.02	181	-5.40 [-12.55, 1.75]	0.13	181
Full follow-up window	-9.01 [-15.78, -2.24]	< 0.01	143	-6.86 [-13.89, 0.17]	0.05	143
Common detected sample	-7.70 [-13.49, -1.92]	< 0.01	126	-5.54 [-11.55, 0.47]	0.07	126
Match ratio, features and size	-7.13 [-13.62, -0.64]	0.03	188	-4.76 [-11.41, 1.88]	0.15	188
Activity-covered reference	-6.82 [-13.33, -0.32]	0.03	183	-4.65 [-11.40, 2.11]	0.17	183
Add activity matching	-4.94 [-12.06, 2.17]	0.17	183	-2.79 [-9.94, 4.36]	0.44	183
Block-covered reference	-8.48 [-15.31, -1.65]	0.01	181	-5.66 [-12.64, 1.32]	0.11	181
Add pre-period blocks	-8.30 [-15.05, -1.55]	0.01	181	-5.33 [-12.29, 1.62]	0.13	181
Unmatched comparison	-7.72 [-13.87, -1.57]	0.01	188	-5.89 [-12.47, 0.68]	0.07	188

Notes: Each cell reports the approximate growth-rate difference 100θ in percentage points for the average of months 0–6 relative to month -1, followed by the equally scaled 95% confidence interval. N is the number of detected projects. TD denotes technical debt in dollars per cumulative recorded feature. The unmatched row is a separate benchmark.

E.4 What the Log Specification Changes

All specifications divide the same burden totals by the cumulative number of feature-labeled commits. The log specification retains positive observed ratios with the required pre-detection history and matches comparison projects on the feature-history variables.

Table 27 separates the outcome definition from sample and matching choices. Its first panel uses the main analysis's weights on projects with observed feature histories. Its second panel uses the feature-history sample and matching weights but keeps the level-ratio outcome. Its final panel changes only the outcome to the log ratio. The level-ratio estimates are not statistically significant, and technical debt is positive in the second panel. The log outcome estimates a difference in growth rates, whereas the level ratio estimates an absolute-ratio response. The table also reports the multiplicity adjustment for the twelve-outcome normalization comparison.

Table 27: What Changes When We Use Log Burden per Feature

Outcome	Effect	95% CI	p	Detected	Clusters
Main-analysis weights; level ratio (percent of pre-period mean)					
Total issues	-2.82	[-18.23, 12.58]	0.71	225	525
Technical debt	-1.01	[-18.22, 16.20]	0.90	225	525
Feature-history sample and weights; level ratio (percent of pre-period mean)					
Total issues	-1.60	[-9.29, 6.08]	0.68	188	403
Technical debt	1.22	[-6.41, 8.84]	0.75	188	403
Same feature-history sample and weights; log ratio (growth difference, pp)					
Total issues	-6.70	[-12.82, -0.58]	0.03	188	403
Technical debt	-5.04	[-11.21, 1.13]	0.10	188	403

All rows use later- or never-detected comparisons and months 0–6 relative to month −1. Level-ratio responses are divided by their own weighted pre-period mean and multiplied by 100. Log-ratio responses and endpoints are multiplied by 100 and reported as approximate growth-rate differences in percentage points. These are distinct units from the percentages of pre-period means in the level-ratio panels. The second and third panels use identical observations and weights. The first-to-second-panel comparison combines sample selection and rematching. The log-ratio total-issue contrast in the twelve-outcome normalization comparison has Holm-adjusted $p = 0.38$; the table reports the individual contrast tests.

E.5 Alternative Measures of Software Functionality

Table 28 distinguishes two ways to measure software functionality. Cumulative feature and changelog declarations record the history of additions. Callable routines, schema and route operations, and interface elements describe the software's current structure. These measures capture different dimensions of functionality.

Table 29 reports alternative level-ratio comparisons, including the secondary interface-action and form measures. The signs and coverage differ across these denominators. Feature-labeled commits provide a recurring declaration of functional development in the project's own history, which motivates the detailed log comparisons above.

Table 28: Measures of Software Functionality

Measure	Recorded object	Comparison
Feature-labeled commits	Cumulative changes explicitly labeled as adding features.	Burden per recorded functional addition.
Changelog additions	Cumulative entries under Added headings in versioned changelogs.	Burden per documented addition.
Public callable routines	Functions exposed for use elsewhere in the software.	Burden per callable routine.
Schema and route operations	Operations declared in interface schemas or resolved web routes.	Burden per exposed operation.
Buttons, actions and forms	Recorded elements through which users interact with software.	Burden per interface element.

Notes: Feature and changelog declarations count cumulative additions. Callable routines, schema and route operations, and interface elements measure the software's current structure. The ratio outcomes divide engineering burden by the corresponding measure.

Table 29: Alternative Normalizations of Engineering Burden

Denominator	Issues: % [95% CI]	*p*	*N*	TD: % [95% CI]	*p*	*N*
Feature-labeled commits	-2.82 [-18.23, 12.58]	0.71	225	-1.01 [-18.22, 16.20]	0.90	225
Changelog additions	7.89 [-2.66, 18.44]	0.14	57	9.45 [-2.70, 21.59]	0.12	57
Schema-defined operations	3.95 [-1.23, 9.14]	0.13	29	4.42 [-0.29, 9.13]	0.06	29
Web-route operations	15.86 [-8.31, 40.03]	0.19	42	12.85 [-6.92, 32.62]	0.20	42
Public callable routines	-36.48 [-97.04, 24.08]	0.23	163	-37.94 [-101.56, 25.68]	0.24	163
Interface buttons	-21.37 [-67.71, 24.96]	0.36	68	-20.06 [-63.37, 23.25]	0.36	68
Interface actions	-8.71 [-36.17, 18.74]	0.53	82	-9.26 [-37.32, 18.80]	0.51	82
Interface forms	5.50 [-0.94, 11.95]	0.09	59	8.78 [1.17, 16.38]	0.02	59

Outcomes are the burden stock divided by the indicated cumulative declaration count or observed interface count. Effects and intervals are percentages of each estimation sample's own pre-detection ratio mean, for months 0–6 relative to month −1, against later- or never-detected projects. *N* counts detected projects. These are level-ratio outcomes; the feature-log outcome changes the proportional quantity being estimated.